



新使命·新研发
New Mission · New R&D

软件架构设计与性能提升

李必信

东南大学计算机科学与工程学院
东南大学软件工程研究所

Python

"

+

*

/

Hello world!

JAVA

C

-

</>

GO

C++



新使命·新研发
New Mission · New R&D

1. 软件系统性能是什么？ 时间、资源利用率、吞吐量、并发用户数、可伸缩性
2. 影响软件系统性能的因素有哪些？ 内因和外因
3. 如何保障或提升软件性能？ 性能工程
4. 软件架构和性能有啥关系？ 架构设计决定全局性能，架构维护提升局部性能
5. 如何利用SA评估性能？ 基于ADL/UML 4+1模型的方法



新使命·新研发
New Mission · New R&D

目录

1. 软件系统性能理解
2. 软件性能工程
3. 软件架构与性能提升
4. 算法原型
5. 结束语



新使命·新研发
New Mission · New R&D

一. 软件系统性能理解

Software performance is also a critical business issue because inadequate system performance can cripple a business that uses the product. Many product development failures result from not being able to meet performance goals. (1988)

R. P. Mikkilineni, AT&T Bell Laboratories
D. F. Utter, AT&T Network Systems



新使命·新研发
New Mission · New R&D

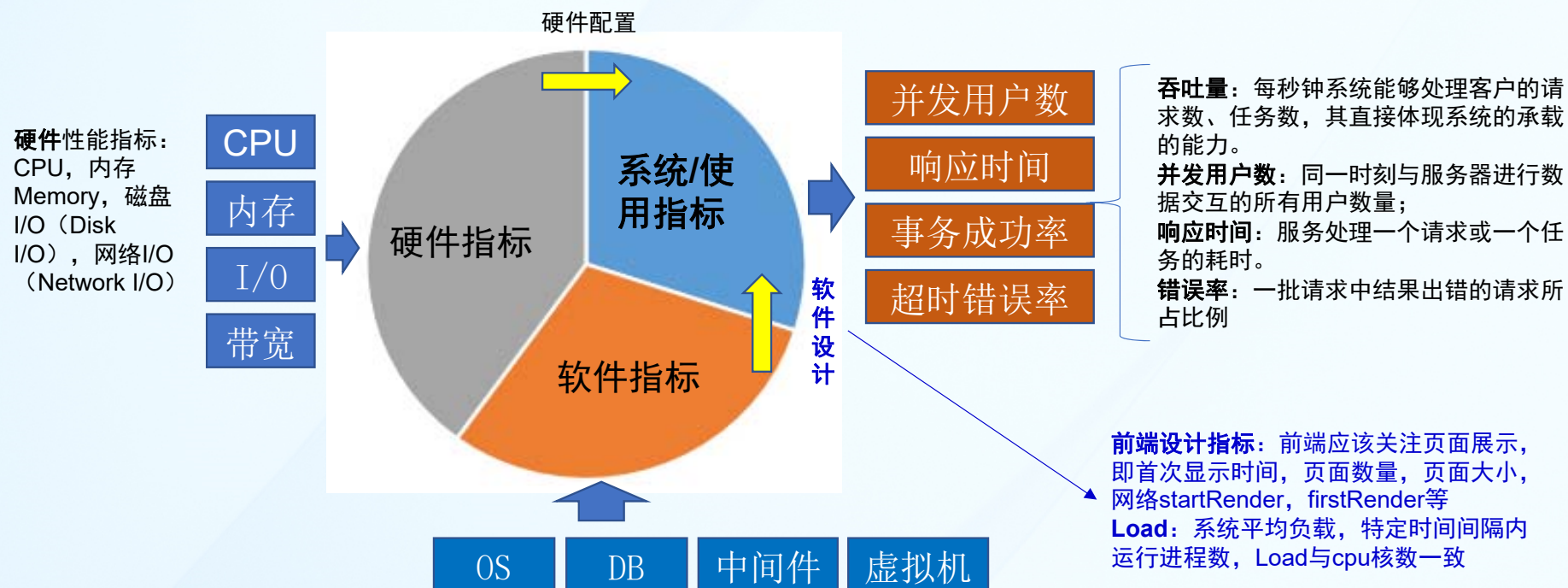
- 1.1 什么是软件性能？
- 1.2 什么会影响软件性能？

1.1 什么是软件性能？

- 软件性能（software performance）是软件在运行过程中表现出来的时间和空间效率与用户需求之间的吻合程度。
 - 从宏观上看，软件性能就是软件运行时完成一功能的响应性特性，以及增加软件功能时保持响应性的能力，即可伸缩性特征。
 - 狭义上讲，软件性能是指软件在尽可能少地占用系统资源的情况下，尽可能高地提高运行速度。内涵涉及**响应时间、系统响应时间和应用延迟时间、吞吐量、并发用户数、资源利用率**等。
 - 广义上讲，软件性能是指软件的质量属性，包括正确性、可靠性、易用性、安全性、可扩展性、兼容性和可移植性等。



新使命·新研发
New Mission · New R&D



- **软件性能指标：中间件：**常用的中间件例如web服务器Tomcat, Weblogic web服务器, JVM (java虚拟机), ThreadPool线程池, JDBC数据驱动；
数据库指标：应关注SQL, 吞吐量, 缓存命中率, 连接数等, 则是关注sql语句执行时间, 以微妙为单位, 吞吐量TPS, 缓存命中率应>=95%；
JVM：Java虚拟机, 为使java的代码可以编译运行在不同的平台上顺畅, 仿真模拟各种计算机来实现



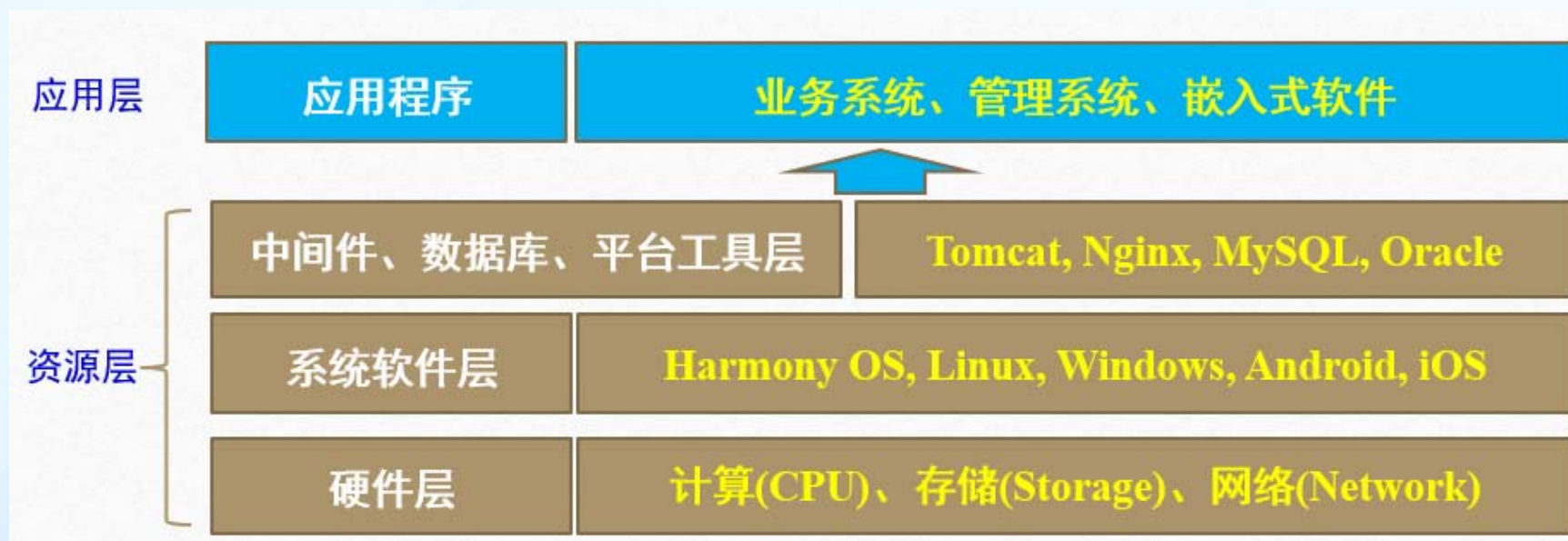
新使命·新研发
New Mission · New R&D

Original System		New System for Scalability	
响应时间 $T=T_N+T_A+T_D$	网络传输时间: T_N	响应时间	网络传输时间
	应用服务器处理时间: T_A		应用服务器处理时间
	数据服务器处理时间: T_D		数据服务器处理时间
吞吐量 $F=N_{VU} * Q/T$	QPS/每秒请求数(Queries Per Second)	吞吐量	QPS
	TPS/每秒事务数(Transactions Per Second)		TPS
	并发数/系统同时处理的Q/T数		并发数
资源利用率 例如, CPU利用率($\leq 75\%$), 其中user, sys, wait, idle分别占多少	CPU ($\leq 75\%$)	资源利用率	CPU
	内存: 物理内存、虚拟内存、页面交换		内存
	带宽: mbps		带宽
	I/O设备: 平均磁盘排队长度是12, ...		I/O设备
.....			



新使命·新研发
New Mission · New R&D

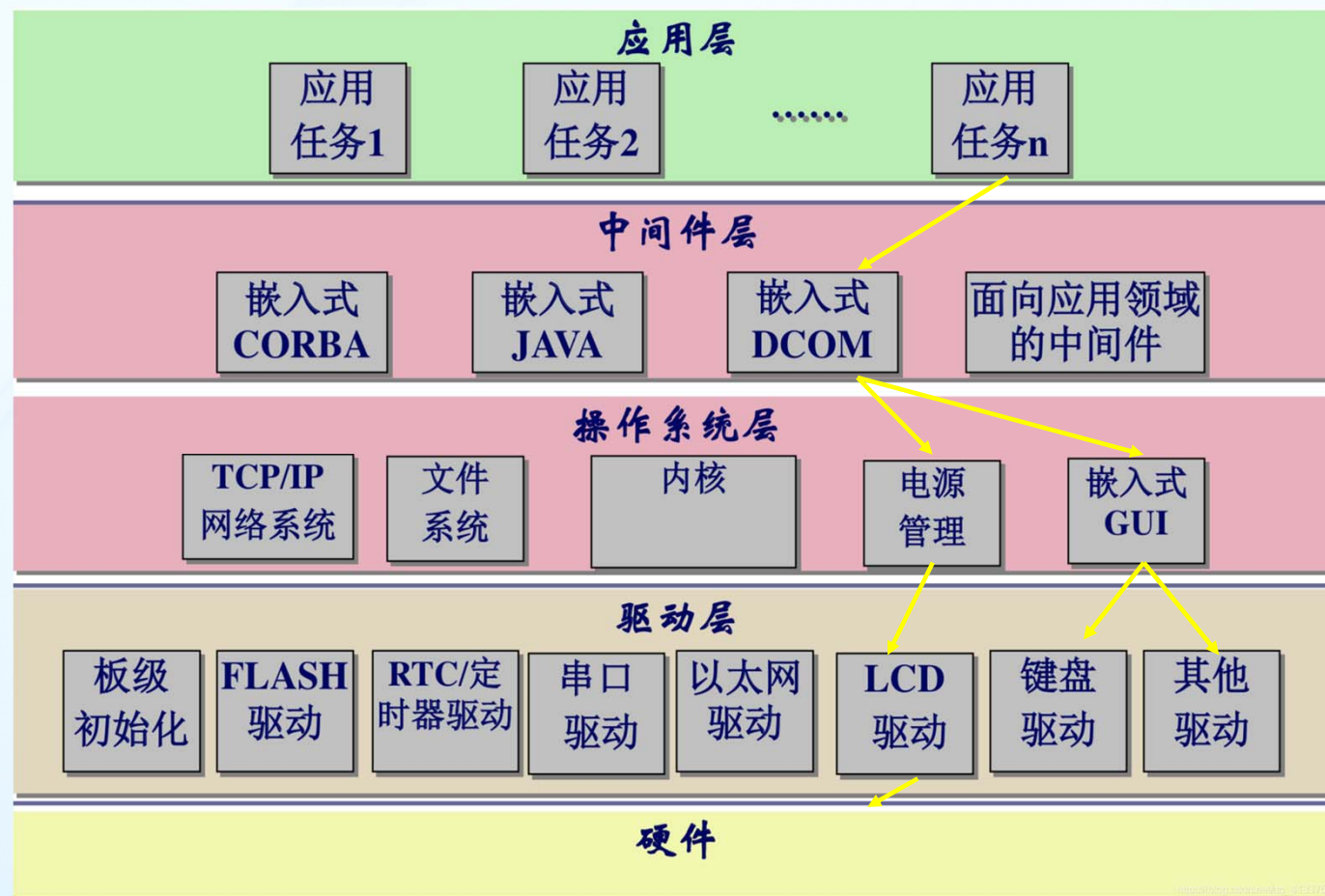
1.2 什么会影响软件性能？





新使命·新研发
New Mission · New R&D

例如：右图是一个包含五层结构的嵌入式软件系统体系架构：硬件层、驱动层、系统软件层、中间件层、应用层。

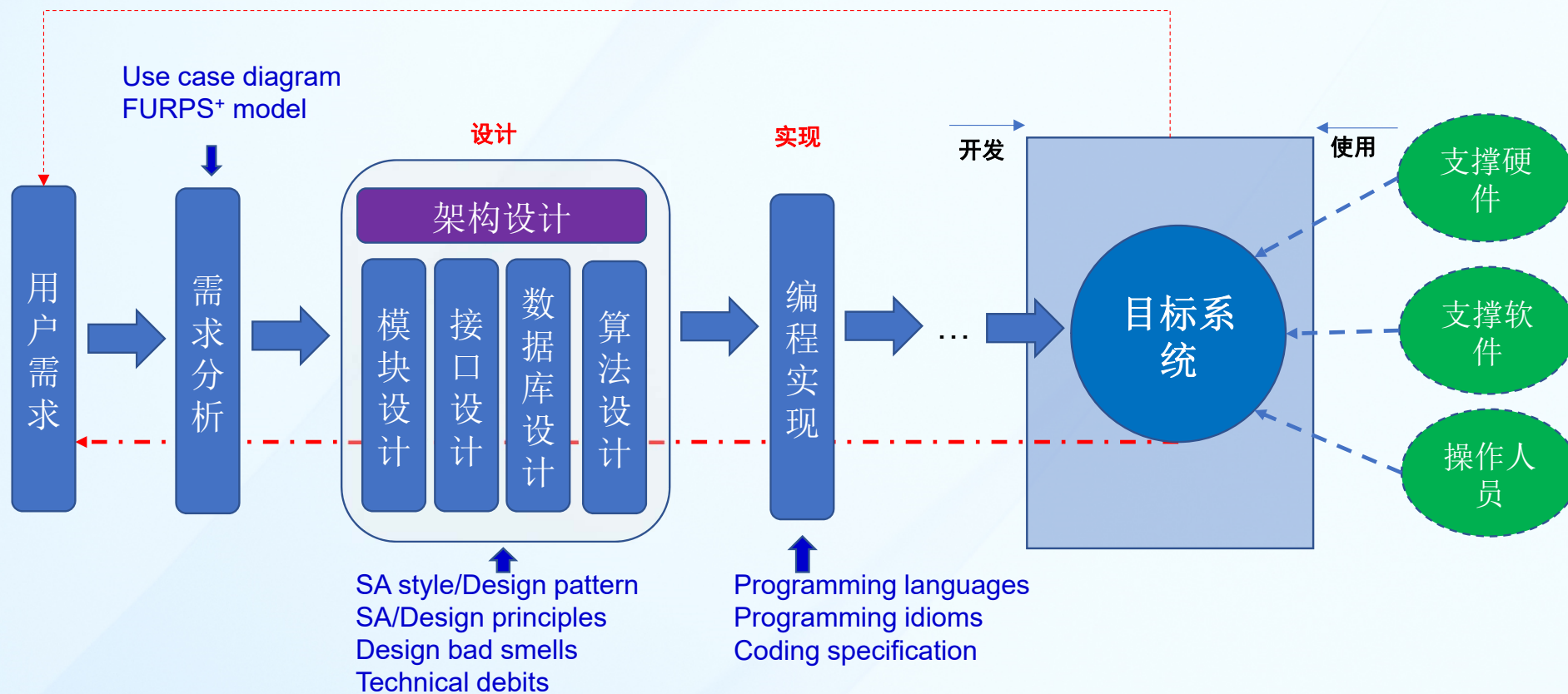


注：图片来源网络



新使命·新研发
New Mission · New R&D

多生命周期的第i次迭代

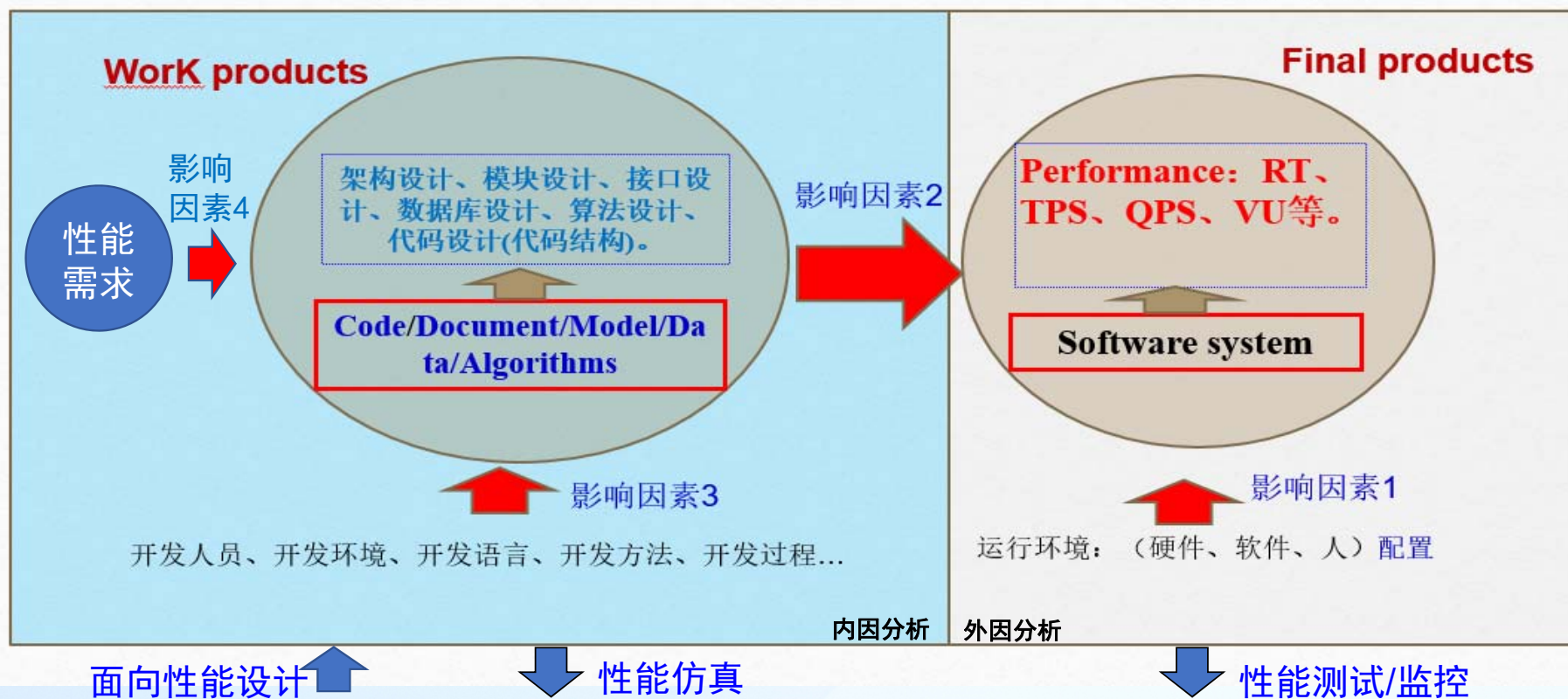




新使命·新研发
New Mission · New R&D

系统开发阶段

系统使用阶段



1.2.1 软件开发过程决定软件性能（内因）

① 用户需求：做正确的事！

- ✓ 如何准确地、正确地描述性能需求？

② 软件设计：正确地做事、更好地做事！性能建模、面向性能的设计！

- ✓ 软件设计是软件开发过程中最具创造性的步骤；同时，软件设计是决定软件系统质量（在ISO25010中指功能正确性、性能、安全性等）好坏的关键步骤：如何设计性能更好的软件？
- ✓ 软件设计包括软件架构设计、模块/组件设计、接口设计、数据库设计、算法设计、程序设计等一系列环节的设计。任何环节的设计不好都会影响软件系统性能。

③ 程序设计又称实现(implementation)由于使用不同的程序设计语言、采用不同程序设计惯用法、不同的程序设计原则以及不同的编码规范等，使得最终实现的软件系统的性能也不同。

- ✓ 如何实现性能更好的软件？

1.2.2 软件运维过程影响软件性能（外因）

④ 软硬件配置：硬件性能、软件性能、兼容性等

- **磁盘 I/O**：磁盘的读写速度远慢于内存的读写速度，系统运行是如果需要等待磁盘I/O的完成，将导致整个系统的性能下降；
- **CPU性能**：应用对CPU的占用时间不同，应用间对CPU的抢占也将导致系统性能受到影响；
- **网络状态**：网络本身存在不确定性，其读写速度可能比磁盘I/O还要慢，所以网络状态也可能成为系统性能的一个瓶颈；
- **数据库读写**：当应用可能进行海量数据的读写时，数据库操作将带来想不到的时间消耗，可能影响整个系统的响应；
- **内存大小**：有时候内存过小可能导致一些操作无法完成，导致系统崩溃；这时也可能为了解决内存不足问题采用分步加载资源到内存中，这有导致了磁盘I/O问题；
- **软件性能和兼容性**：数据库、操作系统、浏览器以及其他工具软件的性能和兼容性等。

⑤ 用户使用习惯：90%的用户喜欢在晚上8-10点之间使用(并发用户数)

二. 软件性能工程

Software performance engineering (SPE) is a systematic, quantitative approach to constructing software systems that meet performance objectives. (2001)

Connie U. Smith
Duke University
L&S Computer Technology, Inc.



新使命·新研发
New Mission · New R&D

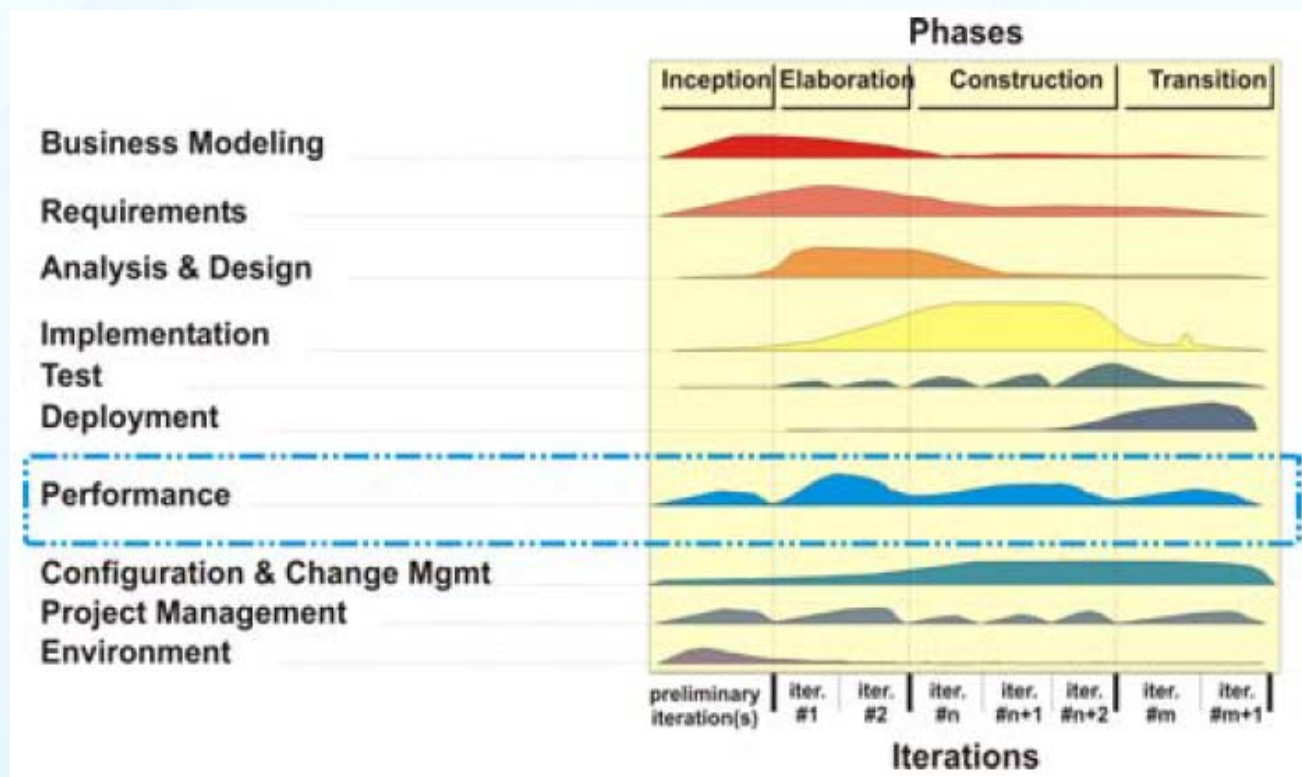
- 2.1 什么是SPE?
- 2.2 SPE的一般过程和任务

2.1 什么是SPE?

- Software performance engineering (SPE) **is a method** for constructing software systems to meet performance objectives[2].
- SPE **is a set of techniques** proposed by Smith that can be used in the software performance management[3,4].

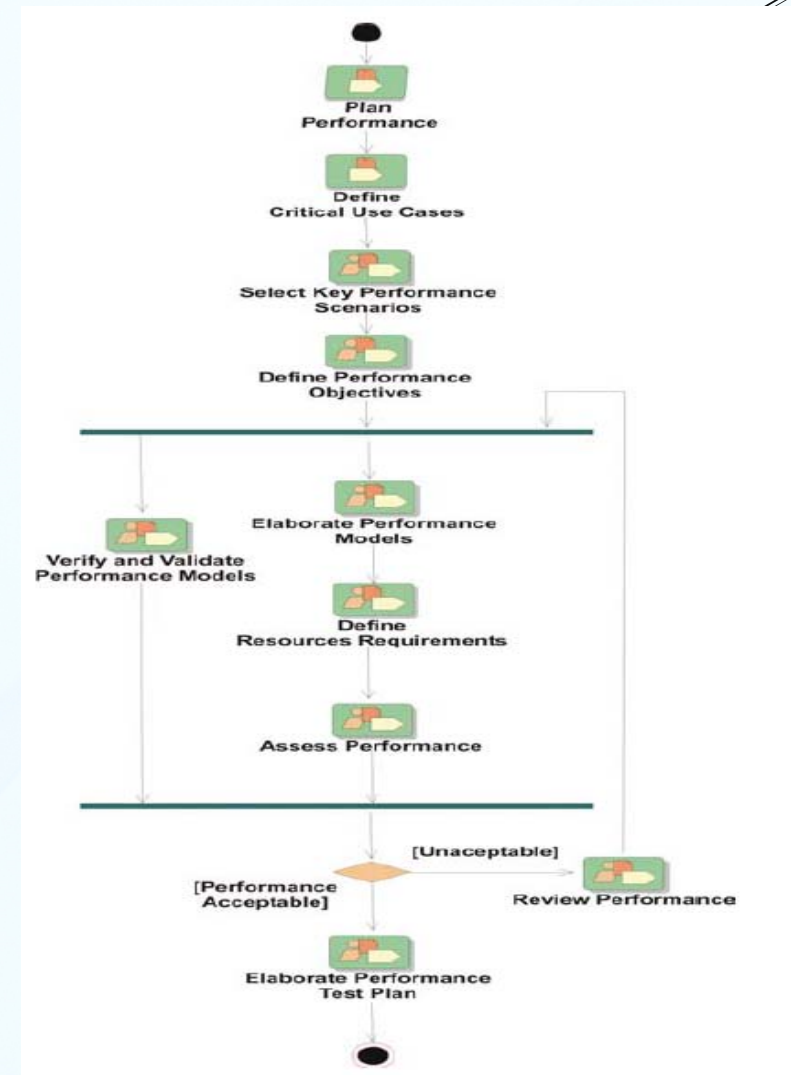
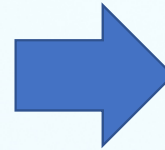
Change software development process for performance

- SPE can be integrated into **any software development process**, such as waterfall or iterative and incremental process[5,6].



例子： RUP Extension For the Software Performance[6]

1. Plan Performance
2. Define Critical Use Cases
3. Select Key Performance Scenarios
4. Define Performance Objectives
5. Elaborate Performance Models
6. Define Resource Requirements
7. Assess Performance
8. Verify and Validate the Performance Models
9. Review Performance
10. Elaborate Performance Test Plan





新使命·新研发
New Mission · New R&D

2.2 SPE的一般过程和任务

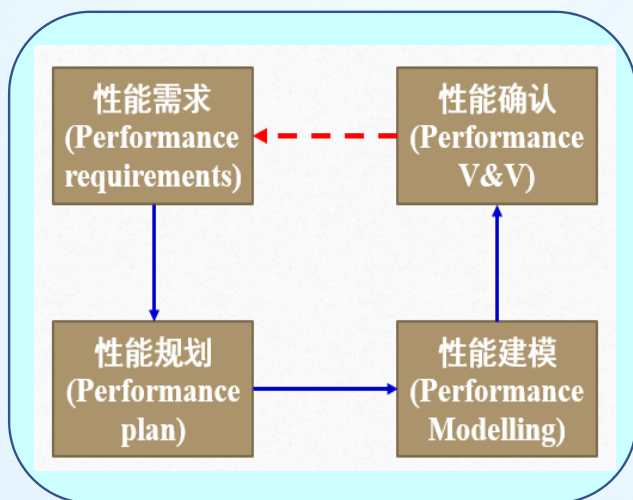
- **性能需求与确认**：确定性能需求内容、范围、指标等。
- **性能设计与评估**：面向性能设计以及通过仿真进行评估优化等。
- **性能实现与测试**：面向性能实现以及通过监控、测试进行评估优化等。
- **性能调优与提升**：通过调整性能参数优化性能，通过设计重构、代码重构、算法重构提升性能，修改性能目标等。

主要阶段	详细步骤	典型方法和工具
性能需求与确认	Performance requirements	USE CASE建模、FURPS+模型 Queueing Network Model, Extended QN, Layered QN Software Execution Model/System Execution Model Process-Algebra Model, Petri-Net Model, Stochastic Processes Model, etc.
	Performance plan	
	Performance modelling	
	Performance V&V	
性能设计与评估	Design for performance	Queueing Network-Based Methodologies; Architectural Pattern-Based Methodologies; Methodologies Based on Trace-Analysis; UML for Performance; Process-Algebra-Based Approaches; Petri-Net-Based Approaches; Methodologies Based on Simulation Methods; A Methodology Based on Stochastic Processes
	Performance simulation	
	Performance measurement	
	Performance evaluation	
性能实现与测试	Implementation for performance	Performance monitoring, Stress Testing, Load Testing, Volume Testing.....1) 常用的性能测试工具: JMeter、LoadRunner...2) 常用的性能监控工具: perf、strace、slabtop、vmstat、dstat、glances
	Performance testing	
	Performance monitoring	
性能调优与提升	Performance tuning/improvement	

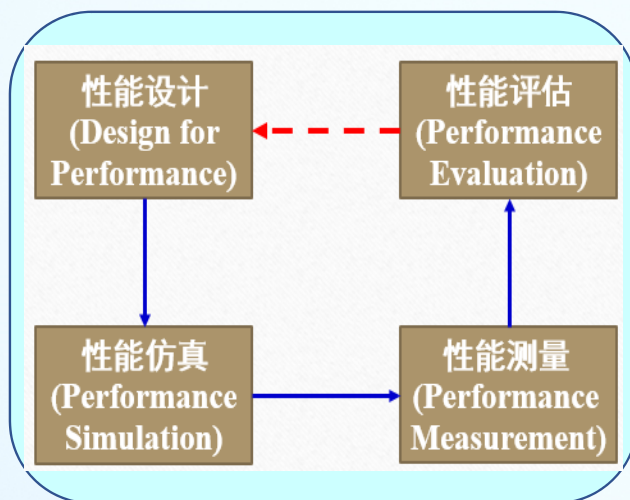


新使命·新研发
New Mission · New R&D

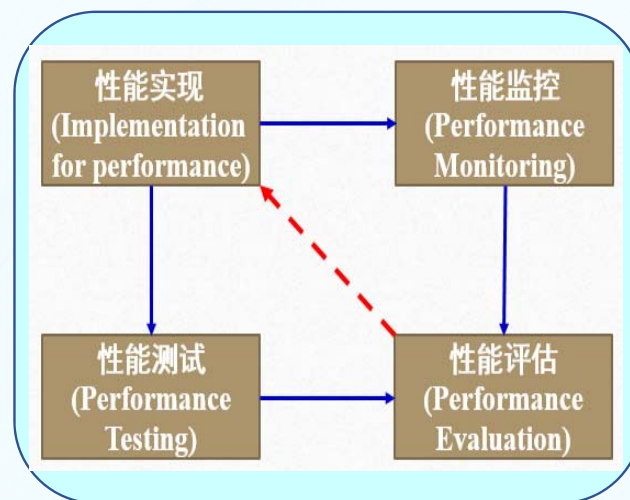
1 性能需求与确认



2 性能设计与评估



3 性能实现与测试



4 性能调优和提升

在资源稳定的情况下, 如何获得想要的性能? 性能工程, 性能建模、面向性能的设计和实现, 性能评估、性能测试等各种性能保障技术。

应用程序层

中间件、数据库、平台工具层

选择合适的中间件、数据库和开发运维平台(IDE)。比如很多web server会用到middleware Tomcat、Nginx, 会用到数据库MySQL、Oracle。

选择合适的操作系统(OS), 目前前后端主流的操作系统是Huawei Harmony OS、Linux, Windows, Android, iOS。同一种操作系统, 不同的发行版本对性能的影响也是比较大。

系统软件层

硬件层

配置给力的处理器(CPU)、存储(Storage)、网络(Network), 计算机硬件资源也主要是这3方面的资源, 现在流行的云计算也主要是这3大资源的虚拟化。

三. 软件架构和性能提升

The Software Architect as the Guardian of System Performance and Scalability

André B. Bondi
*Siemens Corporate Research, Inc.
AT&T Bell Laboratories*



新使命·新研发
New Mission · New R&D

- 3.1 什么是软件架构?
- 3.2 软件架构如何影响性能?
- 3.3 基于软件架构的性能提升原理
- 3.4 基于ADL架构模型的性能提升方法
- 3.5 基于UML架构模型的性能提升方法

3.1 什么是软件架构？

- ◆ A critical issue in the design and construction of any complex software system is its **architecture**; that is, **its gross organization as a collection of interacting components**. *A good architecture can help ensure that a system will satisfy key requirements in such areas as **performance**, reliability, portability, scalability, and interoperability. A bad architecture can be disastrous.* [David Garlan 2000]
- ◆ Software architecture(SA) typically plays a key role as a bridge between requirements and implementation.

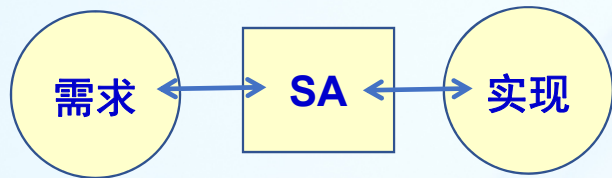


Figure 1: Software Architecture as a Bridge

Path A-D is one in which an ADL is used as the modeling language. Path B-E is one in which UML is used as the modeling notation. Path A-C-E, is one in which an architecture is first represented in an ADL, but then transformed into UML before producing an implementation.

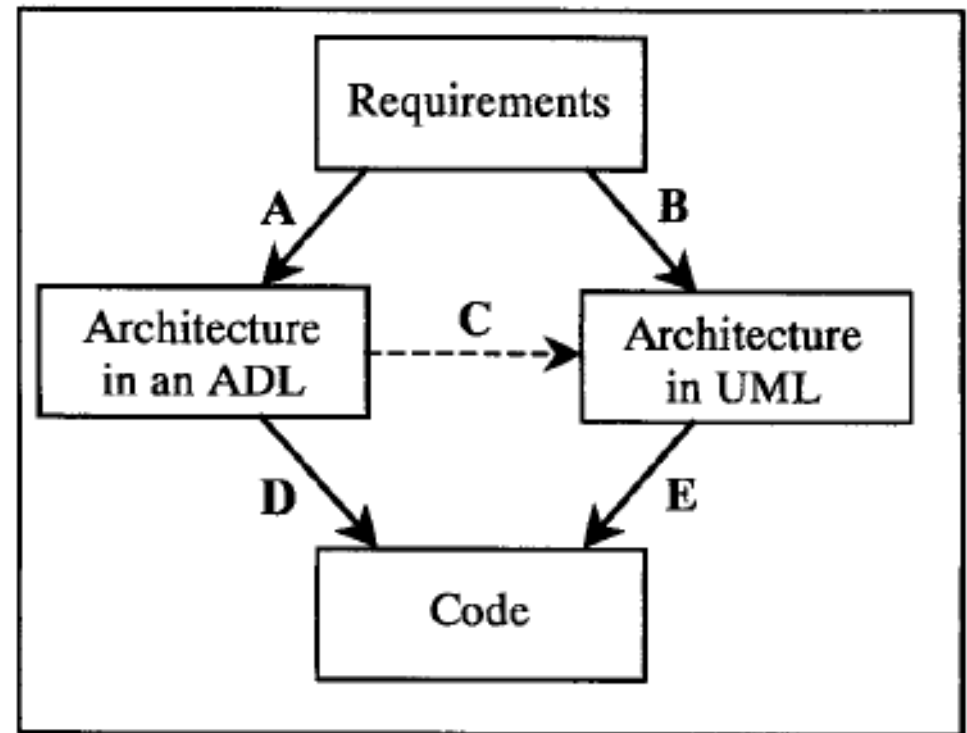
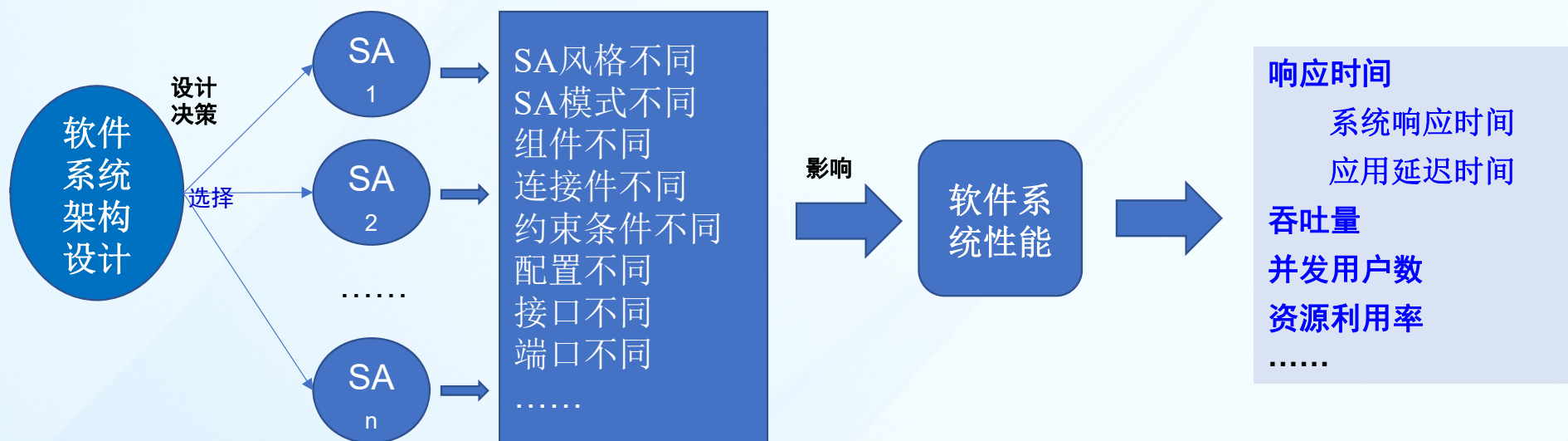


Figure 2: Software Architecture as a Bridge



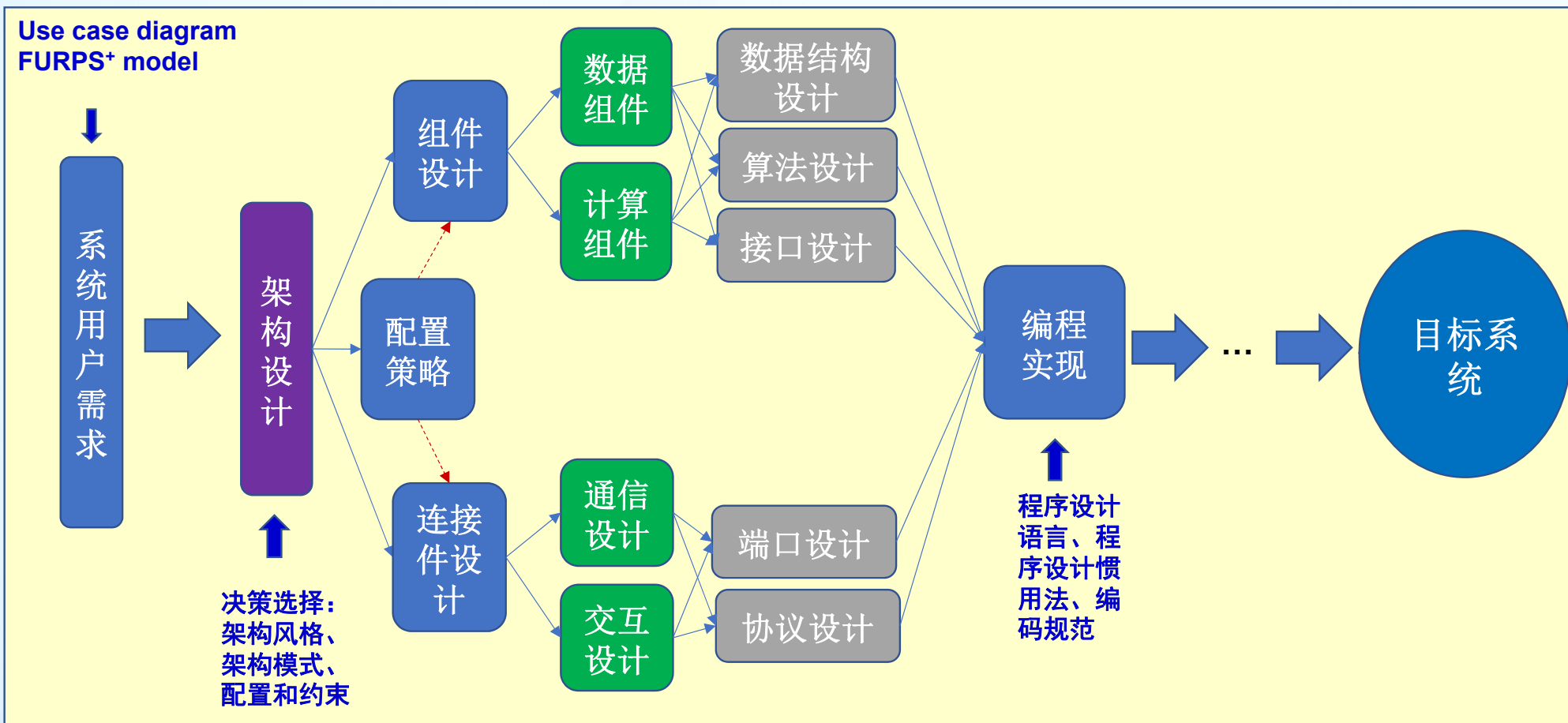
新使命·新研发
New Mission · New R&D

3.2 软件架构如何影响软件系统性能？





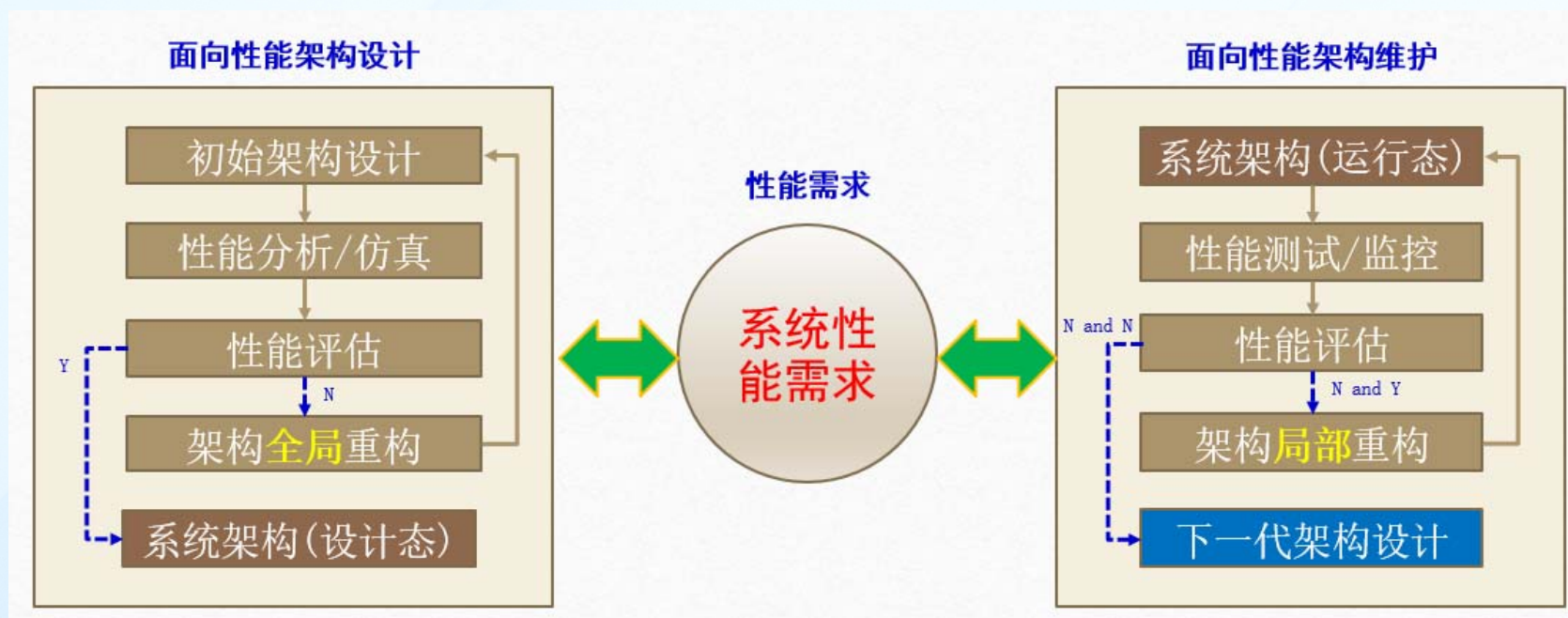
细化的影响传播过程





新使命·新研发
New Mission · New R&D

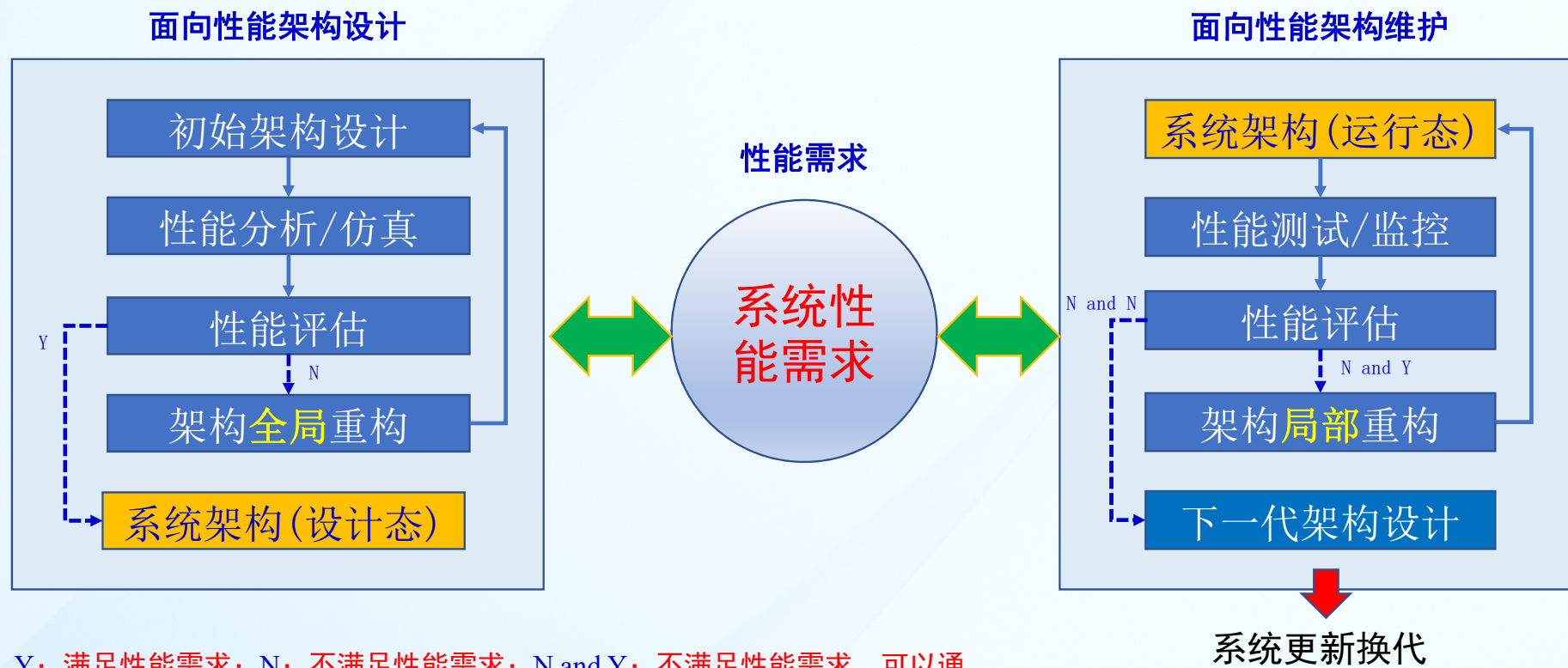
3.3 基于软件架构的性能提升原理





新使命·新研发
New Mission · New R&D

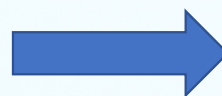
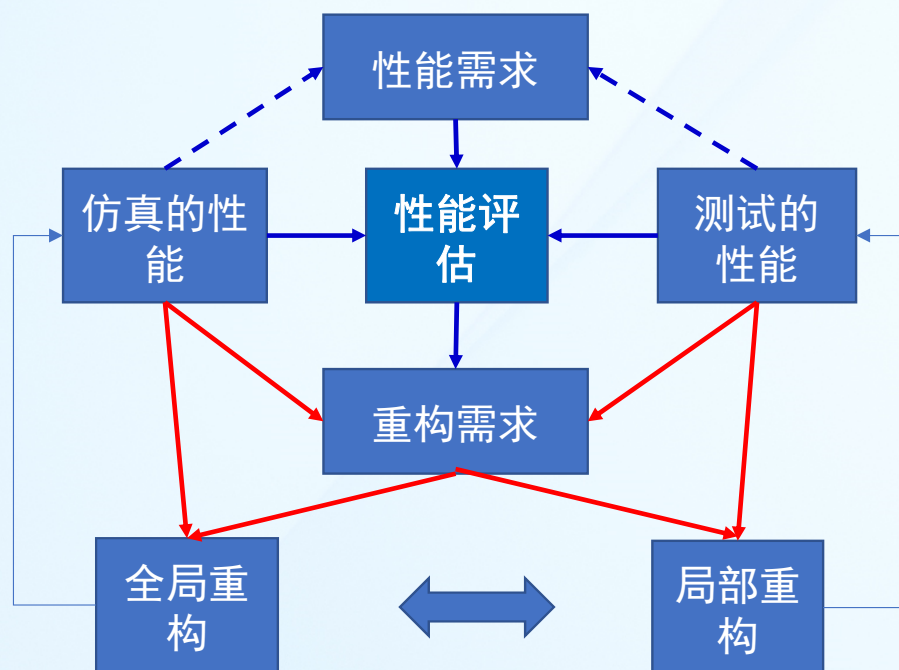
需求：做正确的事；设计：正确地做事、更好地做事！



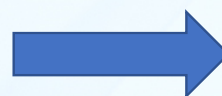
Y: 满足性能需求; N: 不满足性能需求; N and Y: 不满足性能需求, 可以通过局部重构解决; N and N: 不满足性能需求, 但不可通过局部重构解决。



新使命·新研发
New Mission · New R&D



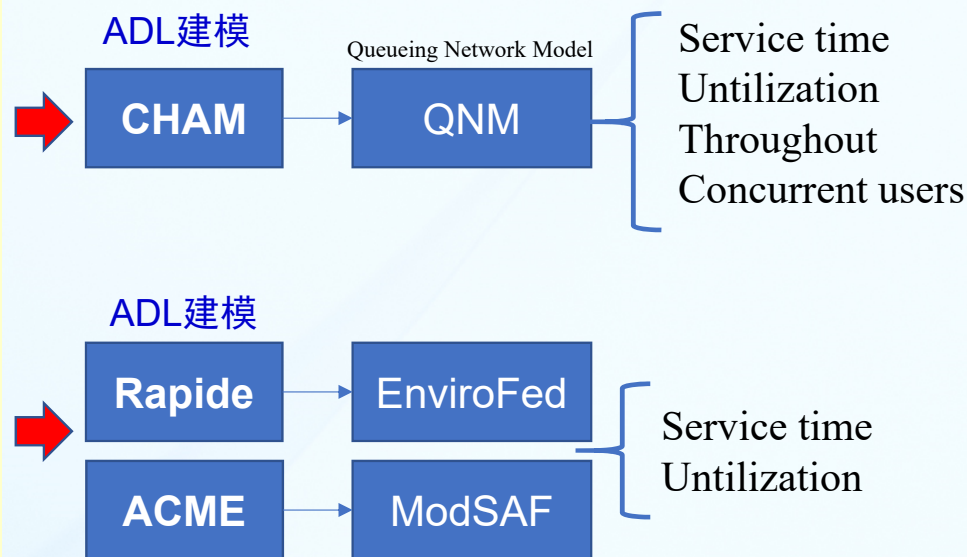
3.4 基于ADL架构模型的性能提升方法



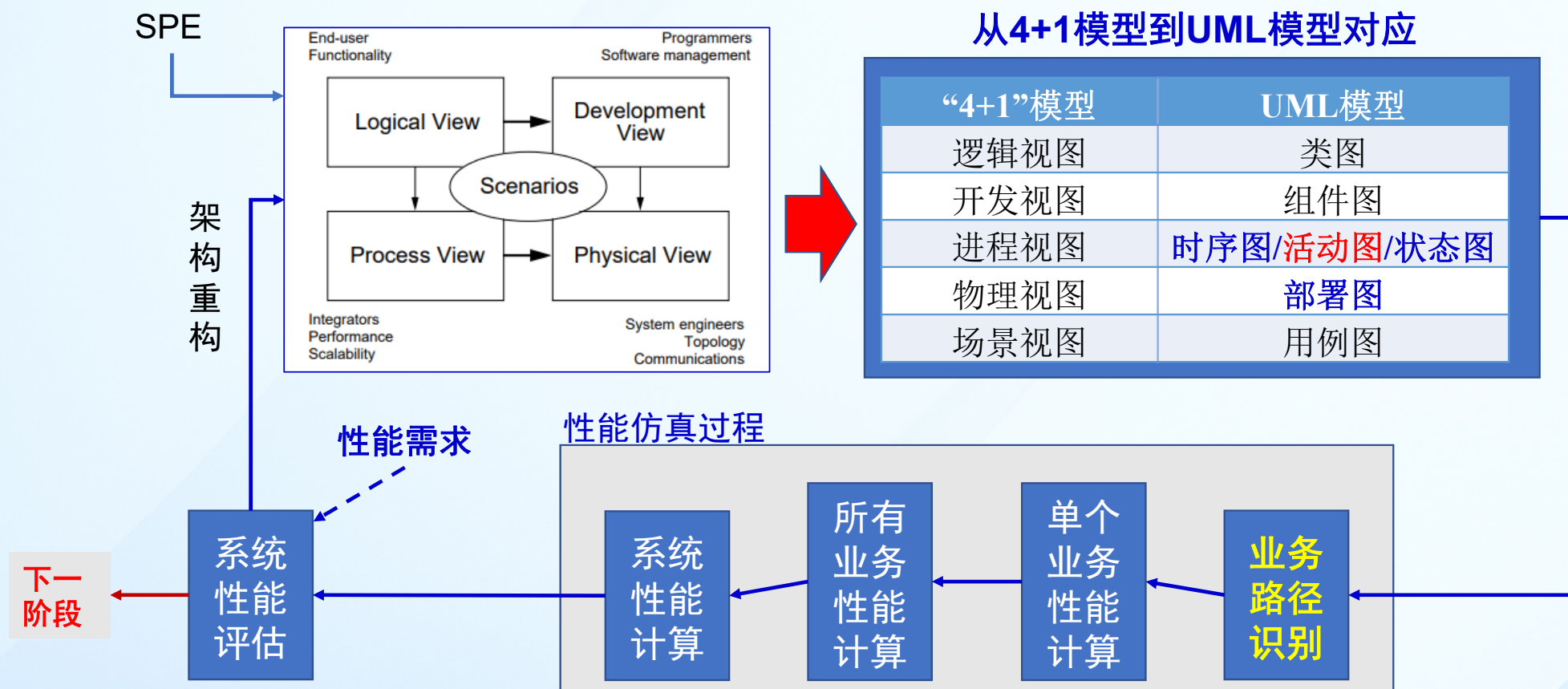
3.5 基于UML架构模型的性能提升方法

3.4 基于ADL架构模型的性能提升方法

- Paola Inverardi, Calogero Mangano and Fabrizio Russo etc. *Performance Evaluation of a Software Architecture: A Case Study*. 9th International Workshop on Software Specification and Design, 1998.
- Mikel D. Petty, Frederic D. McKenzie, and Qingwen Xu. *Using a Software Architecture Description Language to Model the Architecture and Run-Time Performance of a Federate*. 6th IEEE International Workshop on Distributed Simulation and Real-Time ications, 2002.



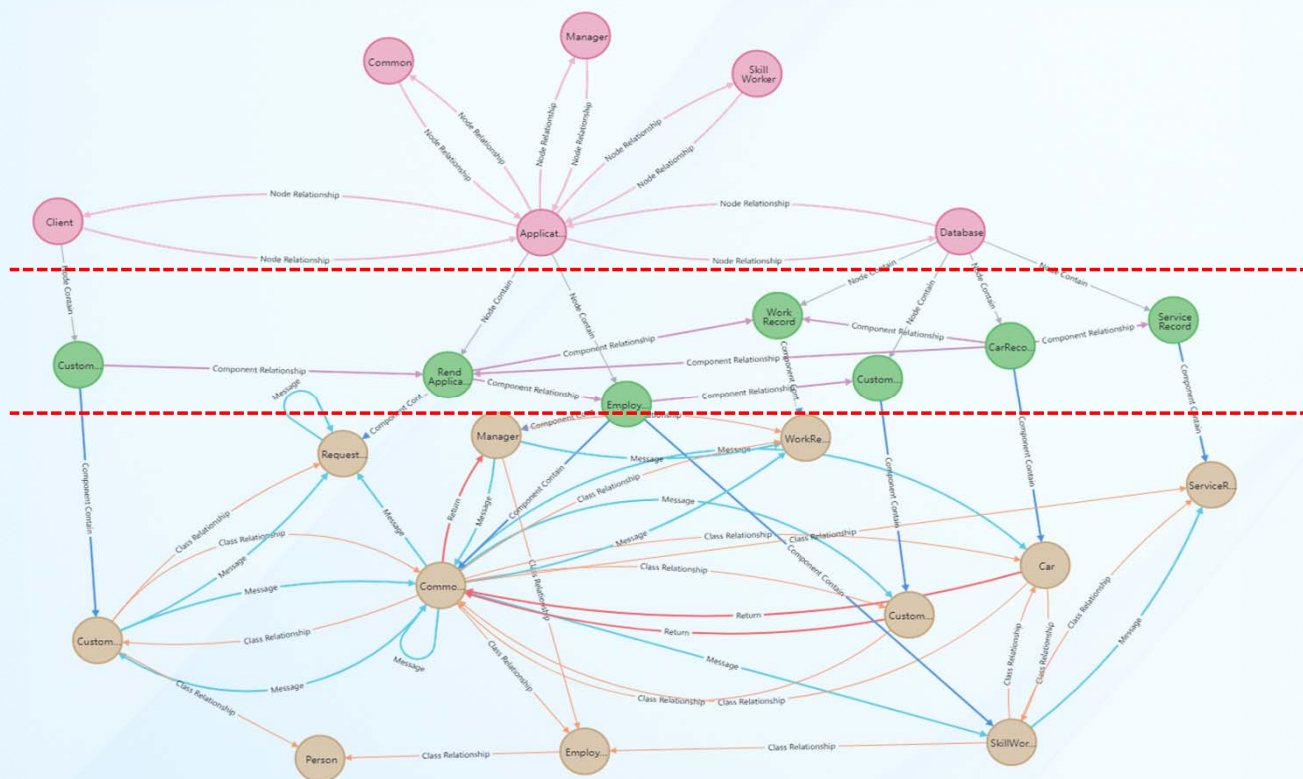
3.5 基于UML架构模型的性能提升方法





新使命·新研发
New Mission · New R&D

MHDG构建



多层次依赖关系图

物理层

部署图

Node Contain

组件层

组件图

Component Contain

逻辑层

类图

时序图



输入设备信息

设备信息：

CPU频率、设备响应时间倍数、设备最小操作时间倍数、设备线程数。

- **CPU频率**：表示CPU每秒运算的次数。
- **设备响应时间倍数**：设备响应所需要的CPU时钟周期数。
- **设备最小操作响应倍数**：原子操作在该设备上所需要的CPU时钟周期数
- **设备线程数**：设备运行同时执行的任务操作数

设备节点	CPU频率 (GHZ)	设备响应时间倍数(10^6)	设备最小操作响应倍数(10^6)	设备线程数
1 Client	4.0	1.0	1.0	5
2 DataBase	4.0	1.0	1.0	5
3 Execution Server	4.0	1.0	1.0	5
4 Interceptor Server	4.0	1.0	1.0	5
5 JAVA	4.0	1.0	1.0	5
6 Maven	4.0	1.0	1.0	5
7 WebUI	4.0	1.0	1.0	5

架构性能评估流程：
1. 模型文件选择
2. 设备信息标注
3. 业务信息标注
4. 架构性能评估
5. 评估结果展示
读入xml文件：C:/U
Desktop/dormitor

提交

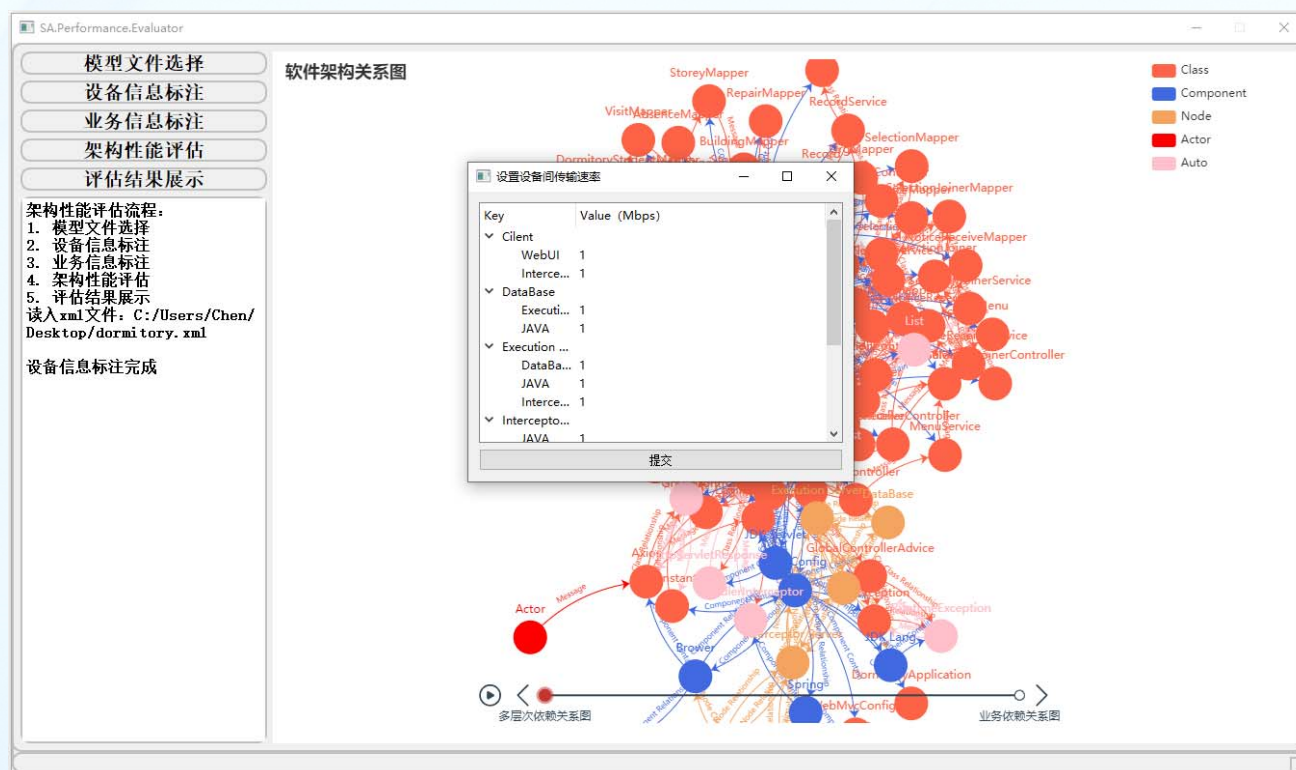


新使命·新研发
New Mission · New R&D

设备间传输速率:

依据设备之间的通信方式和通信设备给定模拟的通信速率。

Mbps (Megabits per second): 表示每秒传输 1,000,000 位 (bit), 即每秒传输的数据量为:
 $1,000,000/8=125,000\text{Byte/s}$





新使命·新研发
New Mission · New R&D

输入业务信息

业务信息:

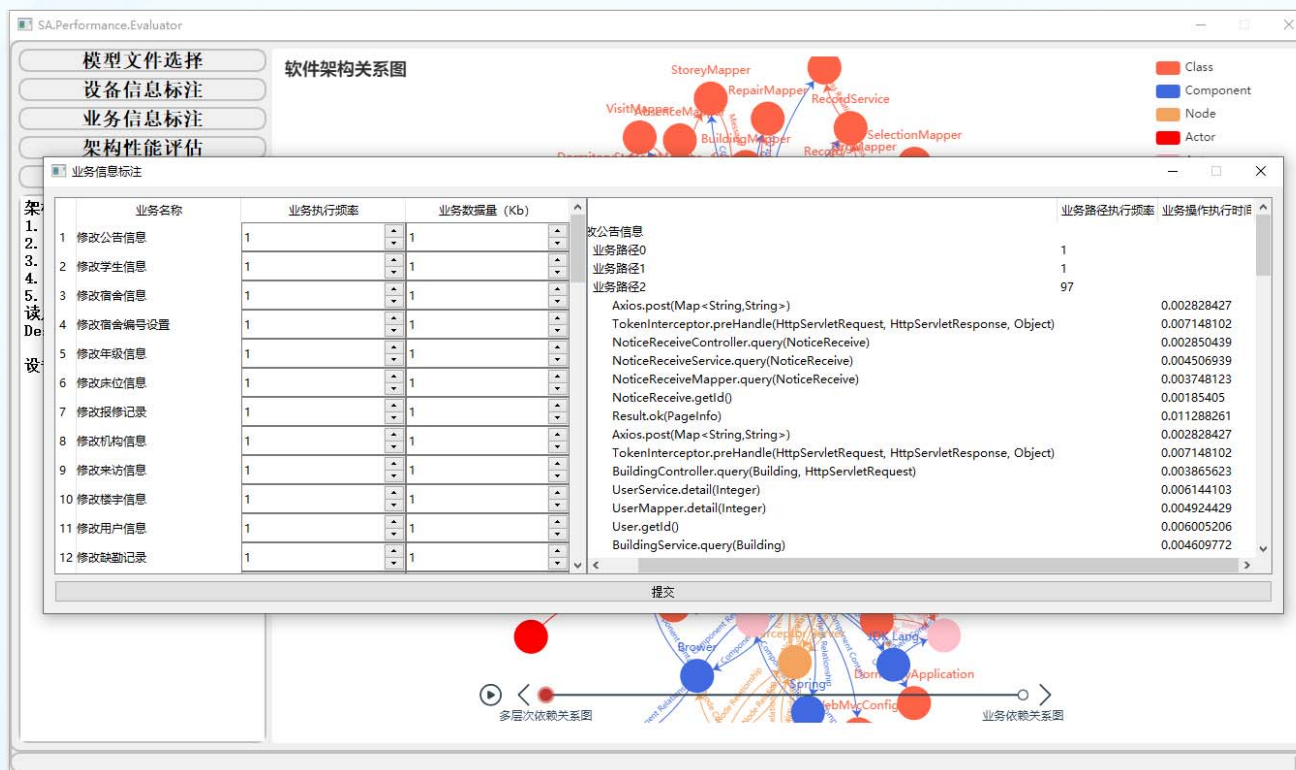
依据业务的执行情况赋予业务相应的业务执行频率、业务数据量、业务路径执行频率和业务操作执行时间。

业务执行频率: 表示该业务在所有系统业务中执行的频率

业务数据量: 表示该业务执行时在系统中传输的数据量(单位为Kb)

业务路径执行频率: 表示用户请求该业务服务时, 执行该条路径的概率

业务操作执行时间: 表示该步业务操作执行所耗费的时间(单位为s)



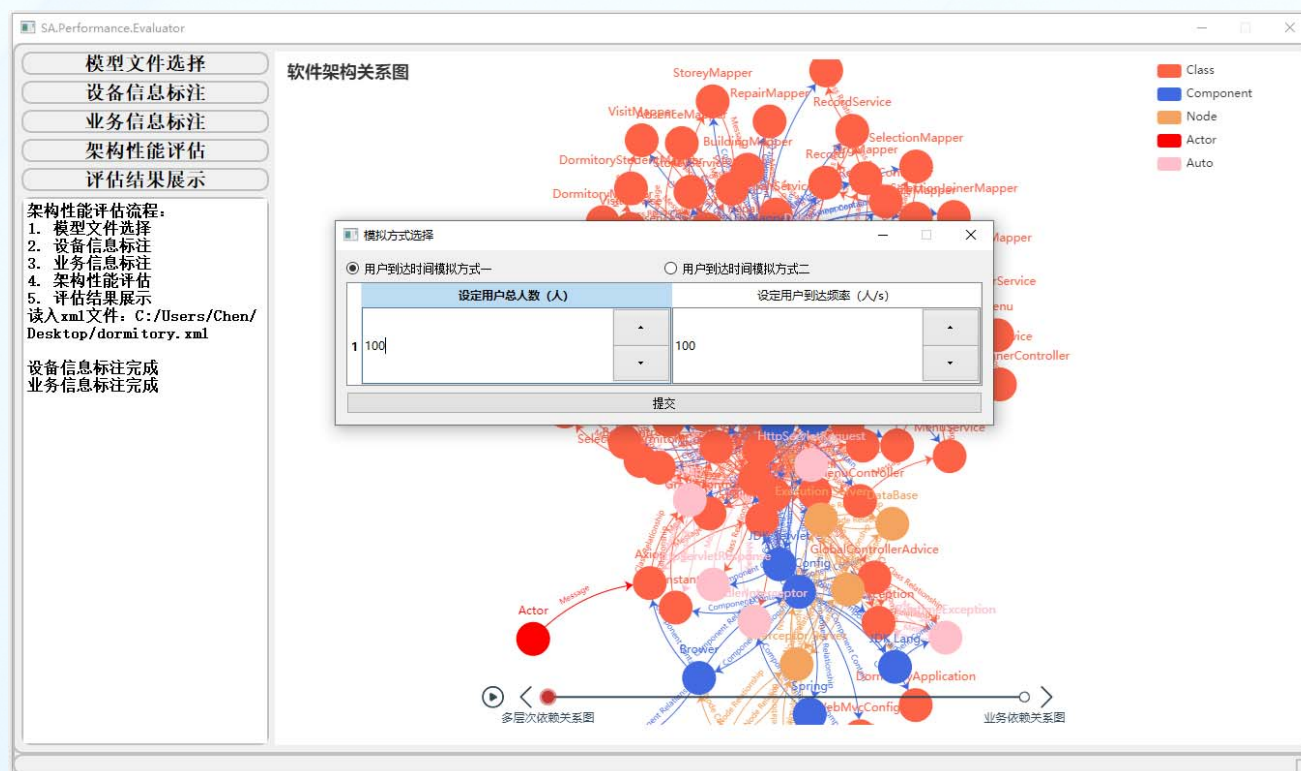


⑤ 输入用户流参数

用户到达时间模拟：假设用户到达的过程是一个泊松过程，用户到达间隔服从 $\frac{1}{\lambda}$ 的指数分布，单位时间内到达用户数服从泊松分布。因此我们设定用户到达频率 λ ，以及模拟过程的用户到达总人数或模拟的总时间来生成用户流。

模拟方式一：通过设定用户到达的总人数，和用户到达频率生成用户到达的时间序列

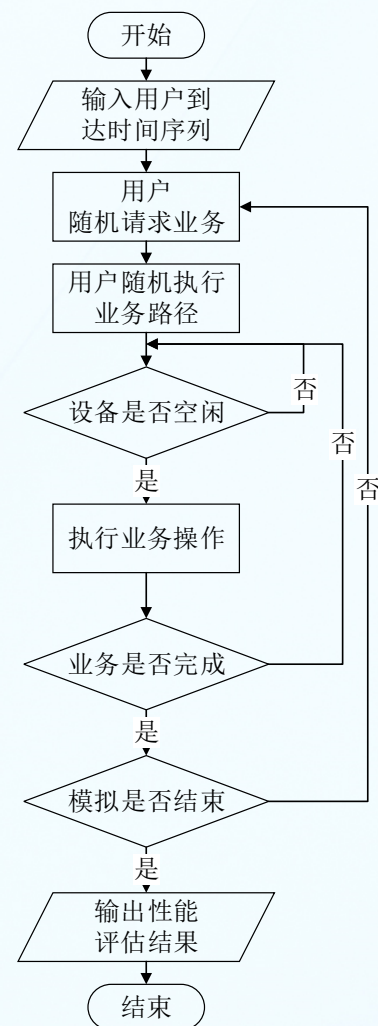
模拟方式二：通过设定用户到达的时间区间长度，和用户到达频率生成用户到达的时间序列



系统性能评估

模拟用户请求系统服务:

1. **输入用户到达时间序列**: 根据用户达到模拟方式与模拟参数计算得到用户到达的时间序列, 构建(用户编号, 到达时间)的序列作为输入。
2. **用户随机请求业务**: 根据业务信息中标注的业务执行频率, 随机为当前用户请求某一业务。
3. **用户随机执行业务路径**: 根据业务信息中标注的业务路径执行频率, 随机为当前用户执行某一业务路径。
4. **执行业务操作**: 一个业务操作需要占用该设备上的一个线程, 当某个设备所有线程都在工作时, 后来的服务请求需要等待某个线程完成上一个操作, 等待队列按照先来先服务原则排队。
5. **输出性能评估结果**: 当系统执行完所有模拟用户请求的业务时, 统计模拟过程中的系统相关性能指标, 并输出。



仿真实验及结果

以学生宿舍管理系统为例

1. 项目概况

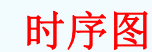
提到宿舍，现在无论是学校，还是工厂，都是在熟悉不过的了，学生宿舍楼，职工教员工，职工宿舍楼等等，每一栋楼房都有很多的房间，每个宿舍分配了多少个床位，住着什么人，那些床位是空的，那些房间是空的，这些都需要人员去维护，曾经的我们，通过纸张登记，查看房间信息，房间床位信息，房间少的情况下还好，我们可以一个个进行比对，看登记信息去查看房间信息，床位信息，但是房间多，登记信息多的情况下，或者有人员进行变动，退房，换房的情况下，可就麻烦了我们的宿舍管理员了，我们的宿舍管理员，需要慢慢的进行查找，浪费了时间成本不说，还容易出错，新来的学生或员工需要等待很长时间才能进去入住，在加上现在楼层在逐渐的向高层发展，人员手动管理已经不能满足社会的需要，因而，社会需要更好的工具去管理这些枯燥的宿舍管理工作，从而宿舍管理系统应用而生。

宿舍管理系统，解决了当前人力成本，时间成本问题，可以快速定位到房间号，宿舍床位号，通过宿舍管理系统，我们只要根据系统的要求去操作，就可以分分钟的时间解决某个宿舍是否满员，可以方便的把人员和宿舍，床位进行绑定，就像我们的身份号似的，都是唯一的，查询起来很方便，相比较起传统的宿舍纸张登记来说，现代化的宿舍管理系统更快捷，更高效，更适应人们快捷化的生活。

现在高校的发展和体制不断的进行了完善，所以学校的服务也逐渐有了变化，对于学校宿舍的管理有了新形势的管理模式。学校根据自身的实际情况，对于学生宿舍生活管理的服务进行改善。有些学校因为管理工作的错误，影响了学校里学生的现况，甚至给整个学校造成了负面的影响，而且现在学校管理问题普遍存在。所以现在学校也在建设信息化管理系统，这种管理系统，可以为学校的管理工作和宿舍管理工作带来很好的效果。



	类	组件	物理节点	用例	活动图	时序图
数量	93	11	7	79	60	60



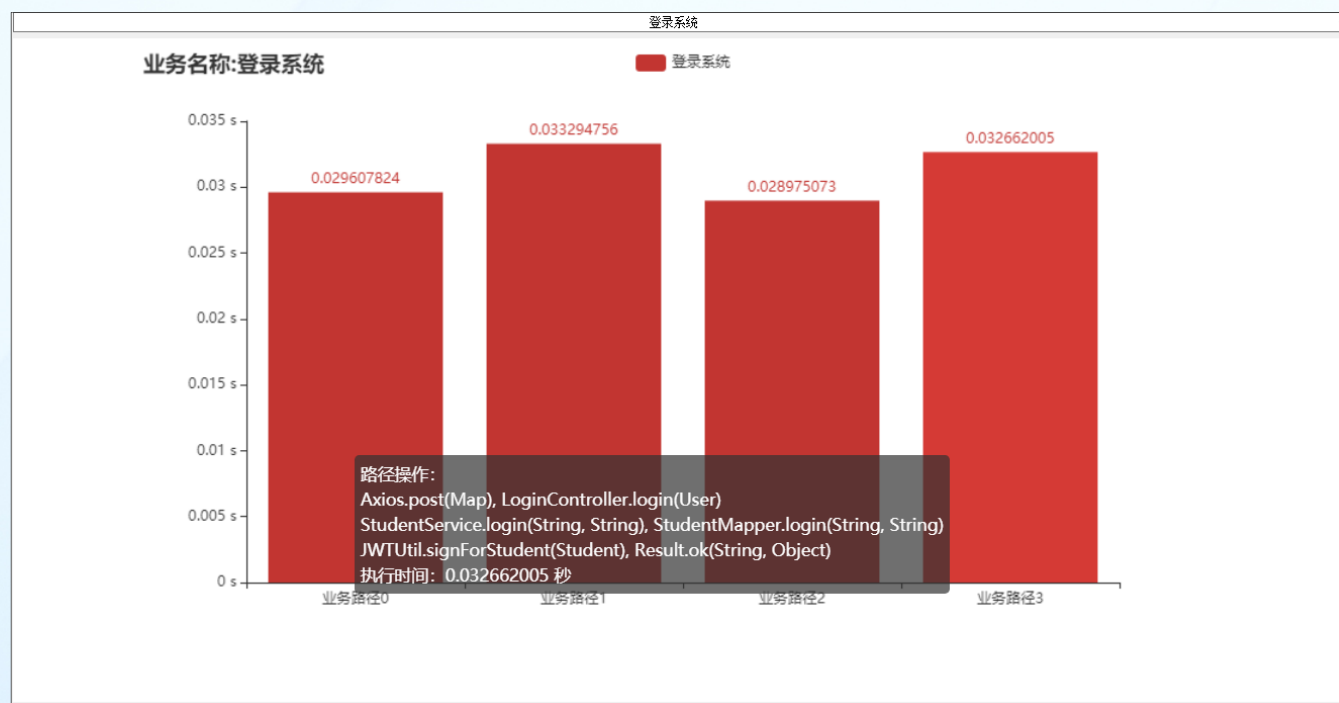


新使命·新研发
New Mission · New R&D

3. 性能评估结果

(3.1) 单个业务的性能展示

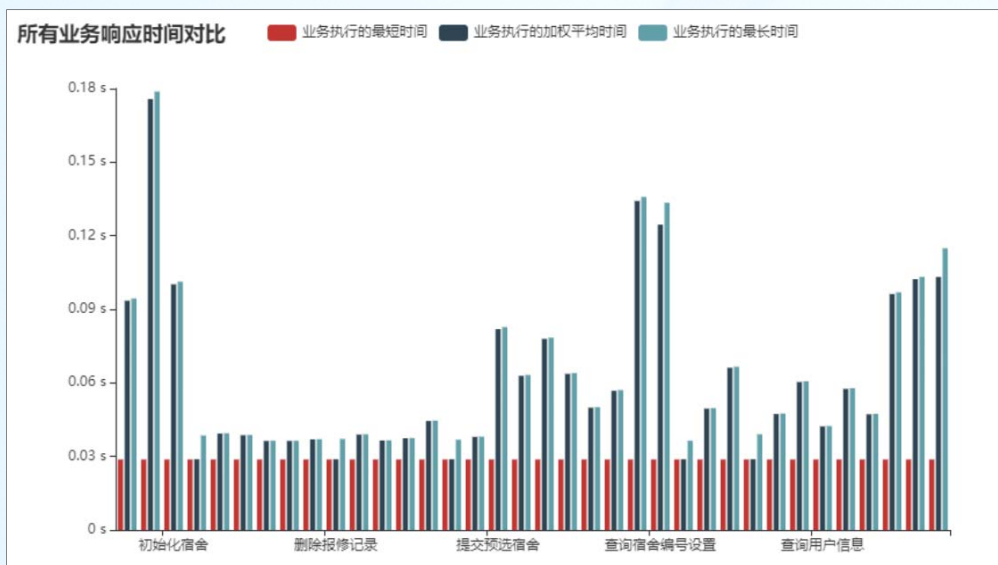
“登录系统”业务不同路径响应时间



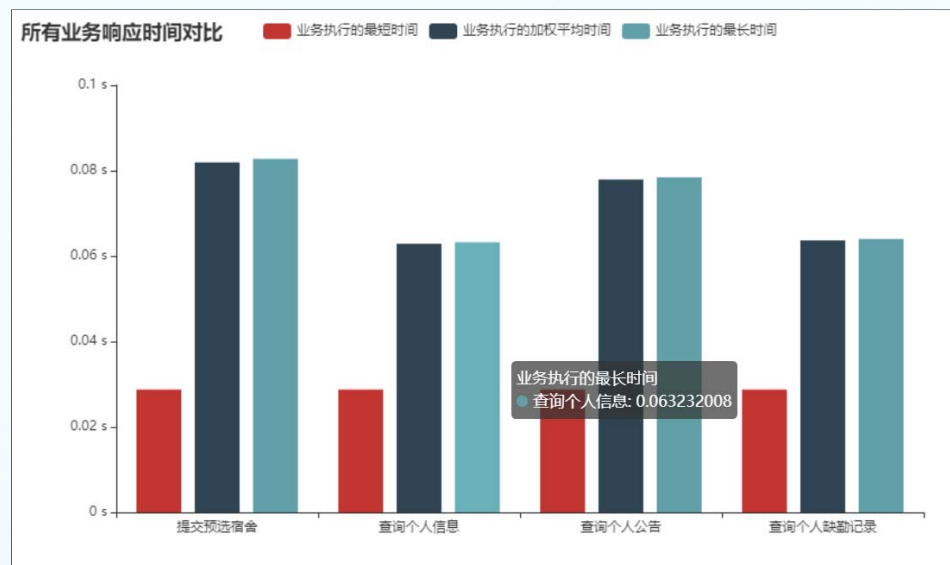


新使命·新研发
New Mission · New R&D

(3.2) 所有业务的性能展示



所有业务性能概况

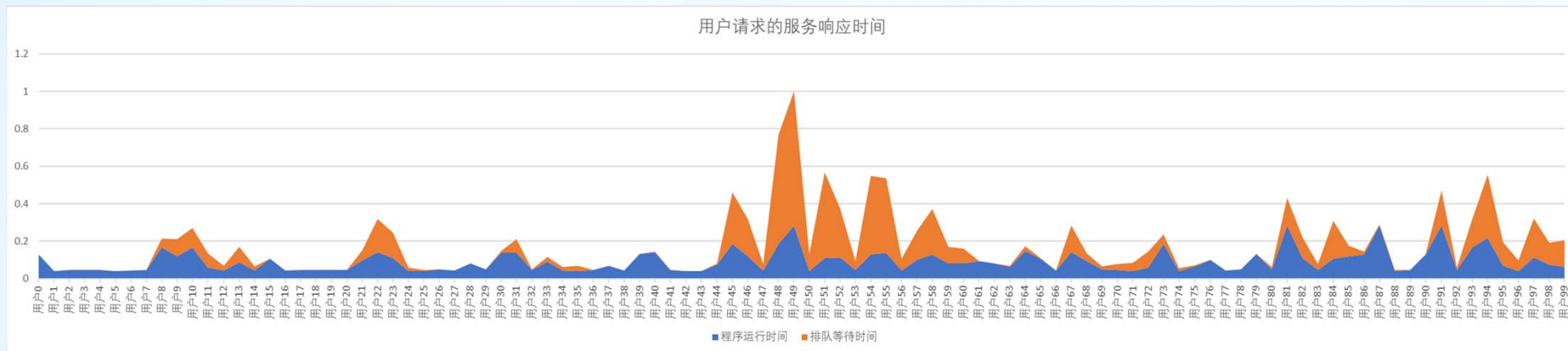


所有业务性能详情

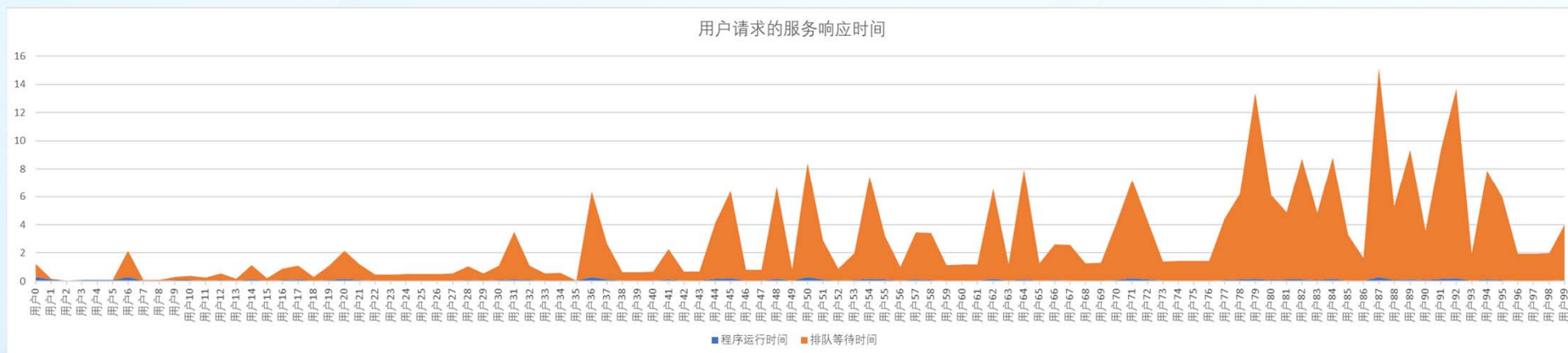
(3.3) 模拟用户达到情况下的系统性能展示



新使命·新研发
New Mission · New R&D



100用户，到达速率为10人/秒



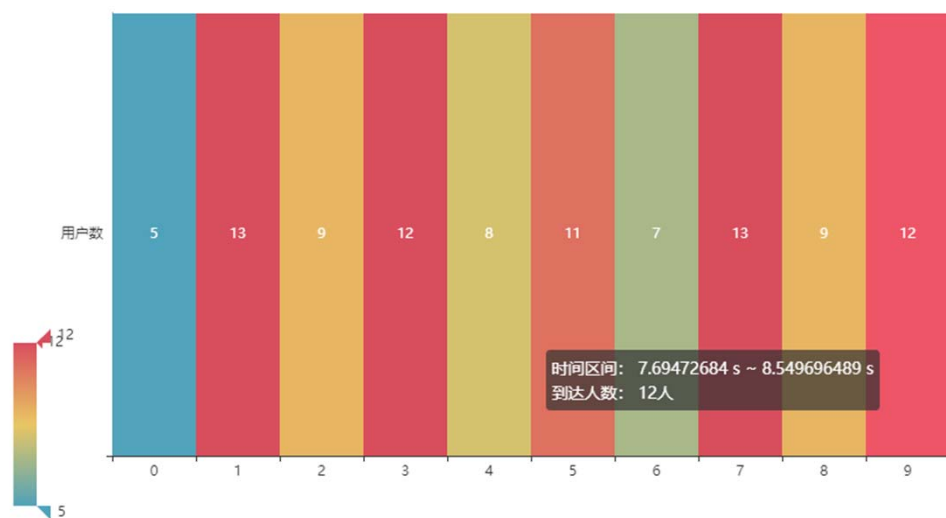
100用户，到达速率为100人/秒



新使命·新研发
New Mission · New R&D

用户到达热力图

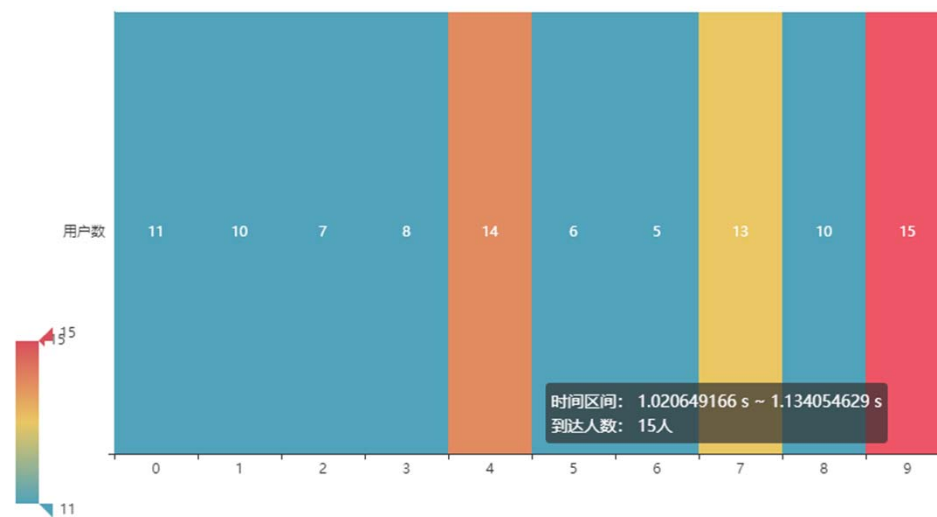
用户到达密度



100用户，到达速率为10人/秒

用户到达热力图

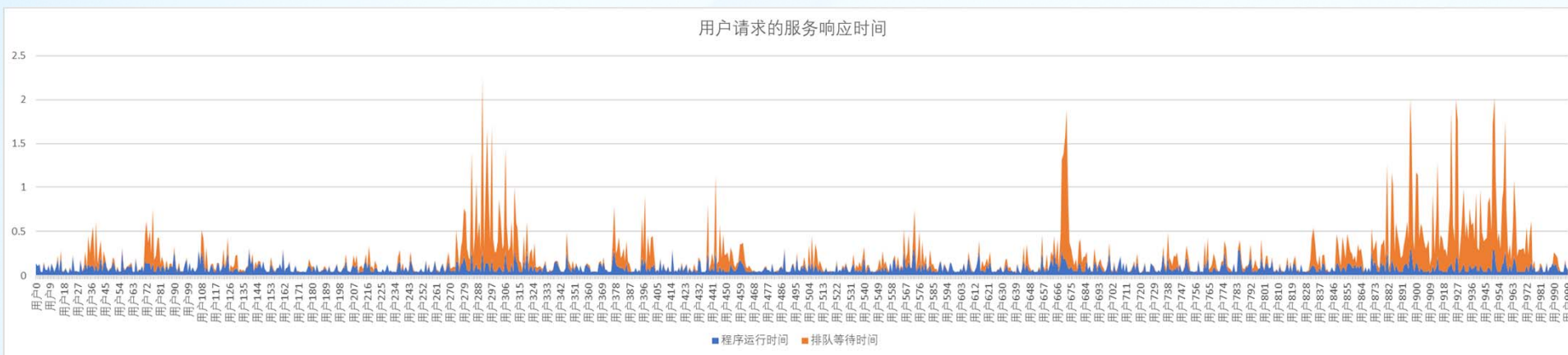
用户到达密度



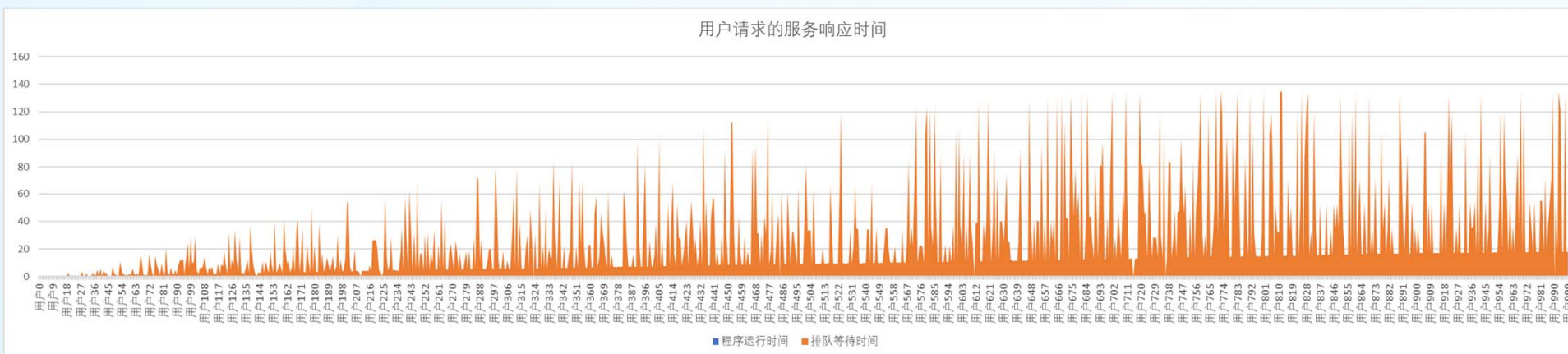
100用户，到达速率为100人/秒



新使命·新研发
New Mission · New R&D



1000用户，到达速率为10人/秒



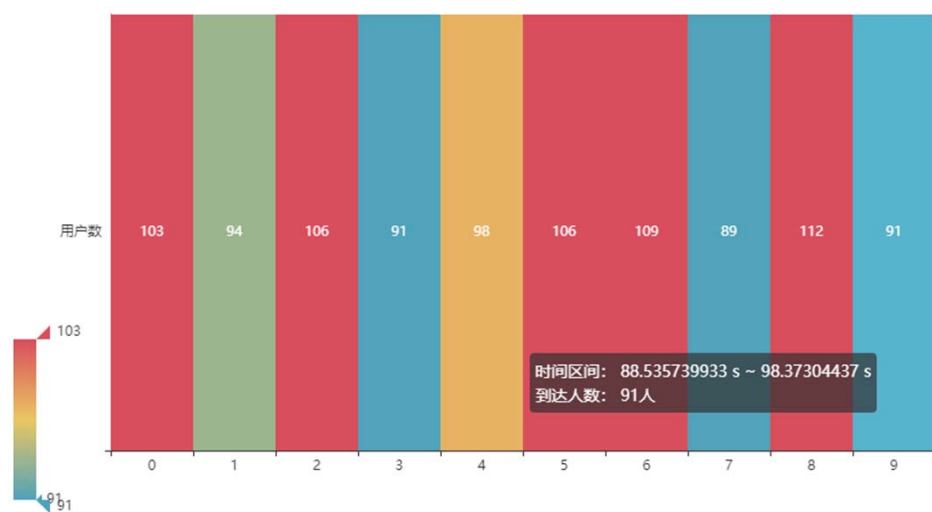
1000用户，到达速率为100人/秒



新使命·新研发
New Mission · New R&D

用户到达热力图

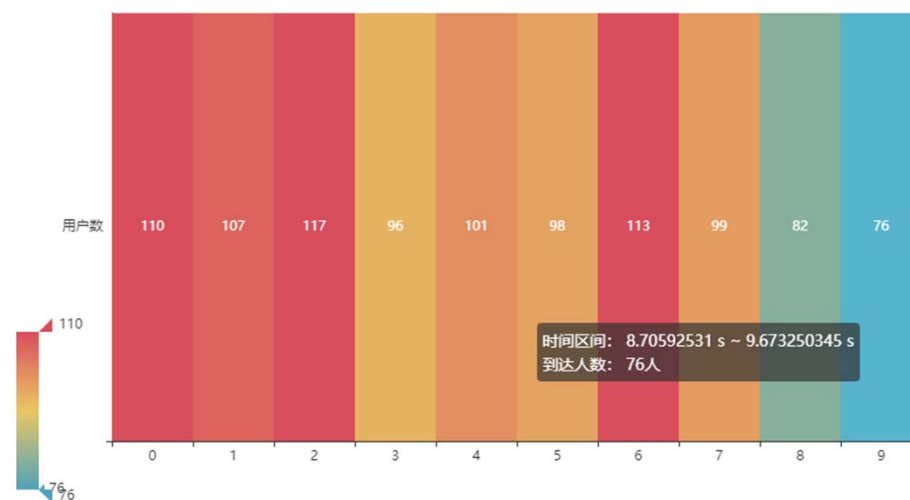
用户到达密度



1000用户，到达速率为10人/秒

用户到达热力图

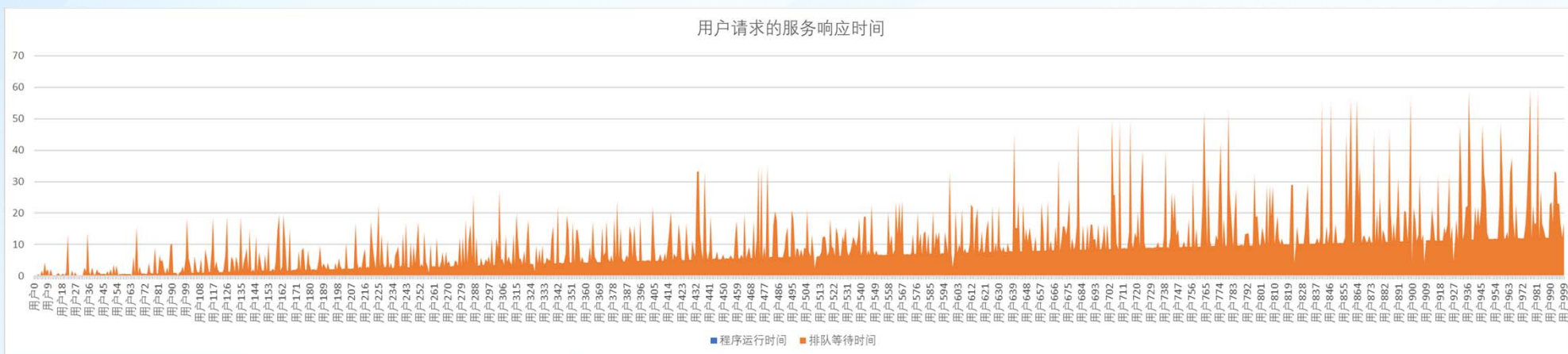
用户到达密度



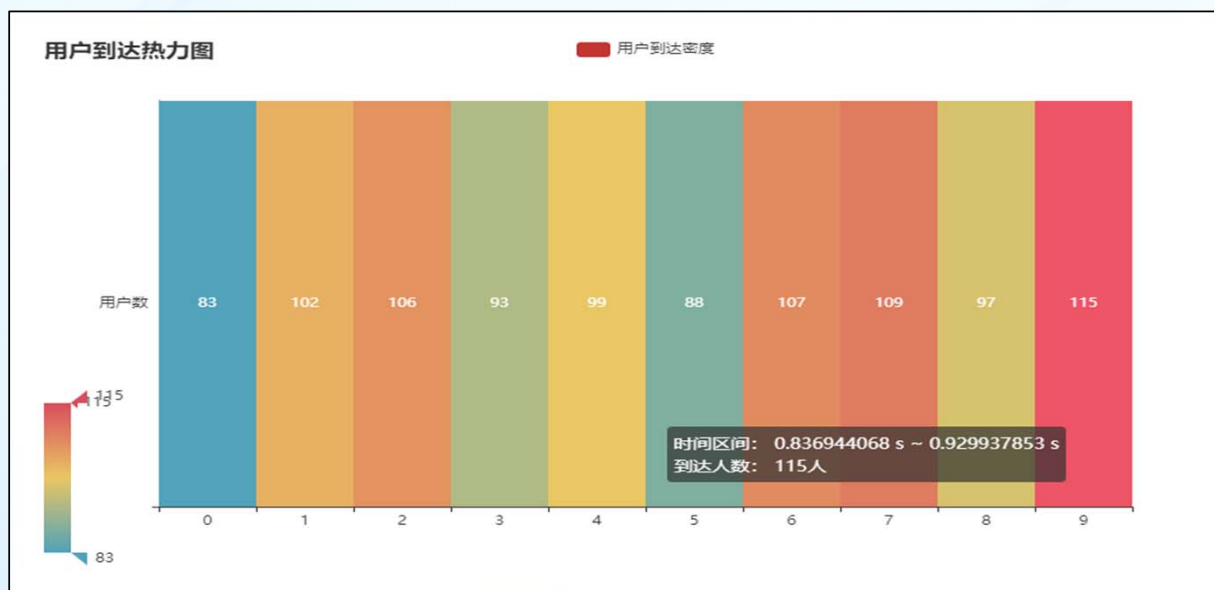
1000用户，到达速率为100人/秒



新使命·新研发
New Mission · New R&D



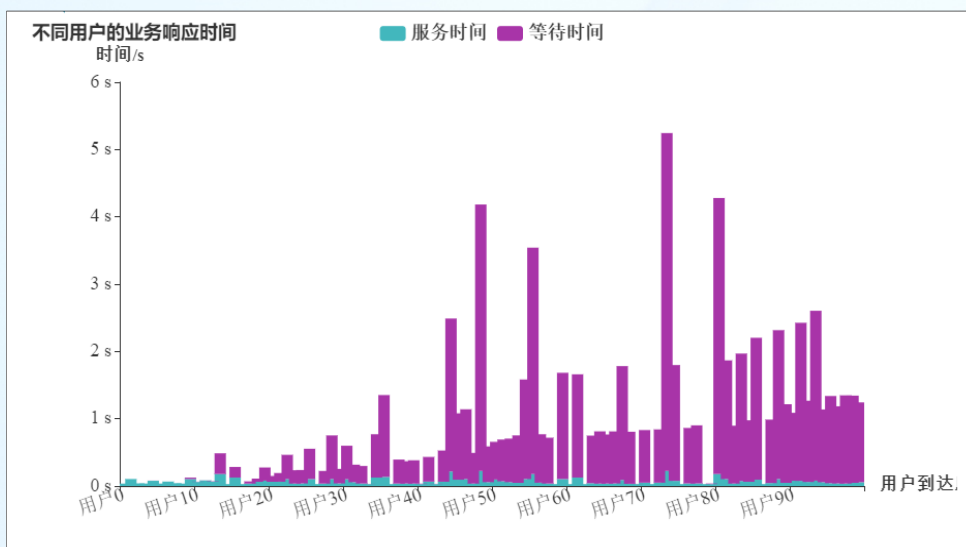
1000用户，到达速率为1000人/秒



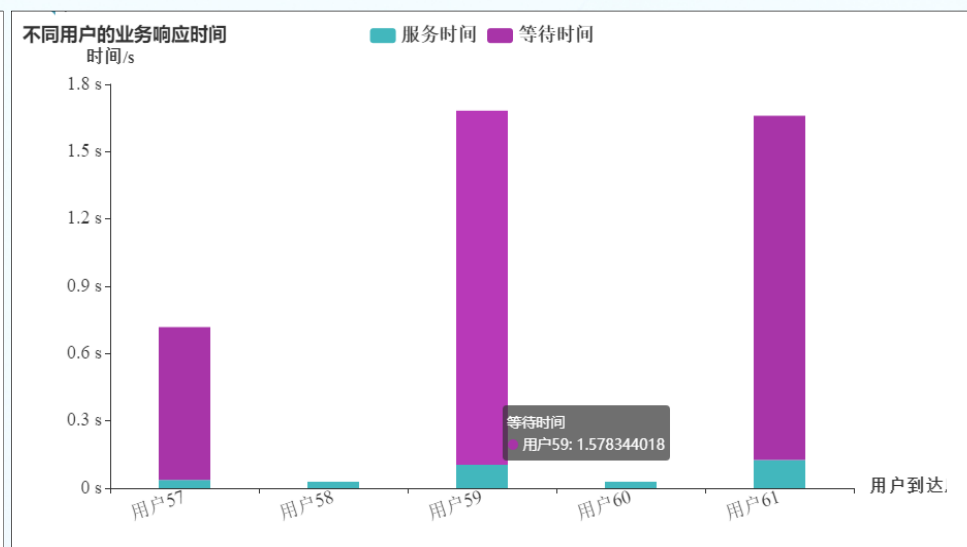
1000用户，到达速率为1000人/秒



新使命·新研发
New Mission · New R&D



用户请求业务响应时间概况

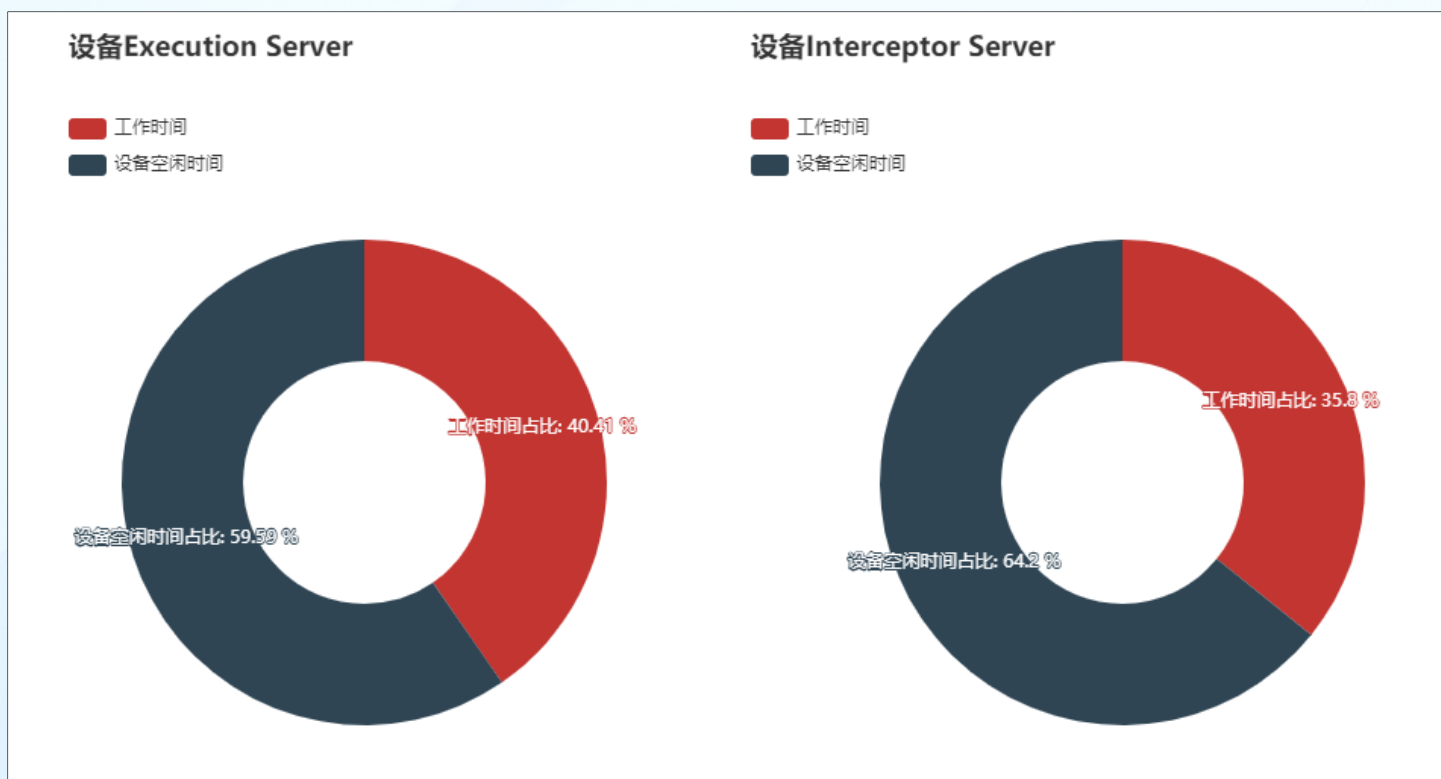


用户请求业务响应时间详情



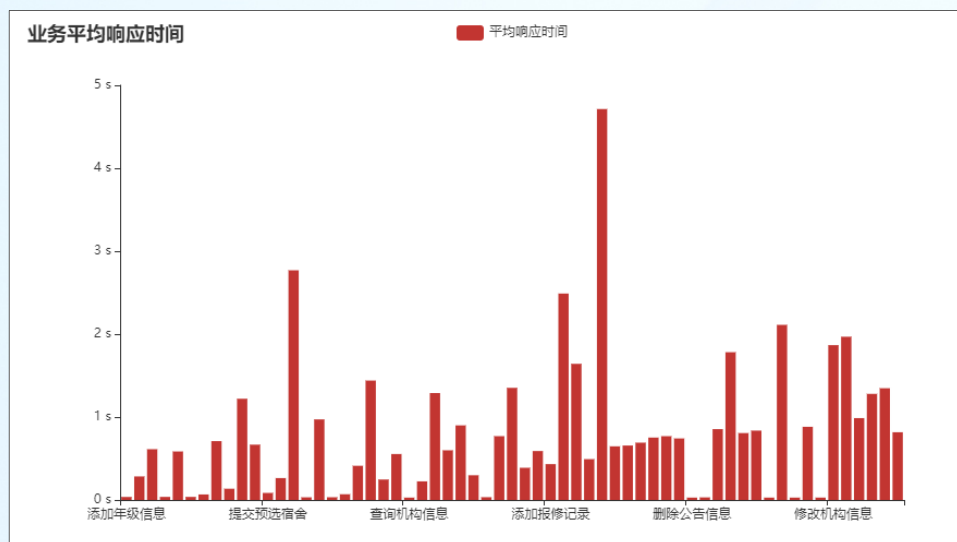
新使命·新研发
New Mission · New R&D

设备工作时间占比展示（部分）

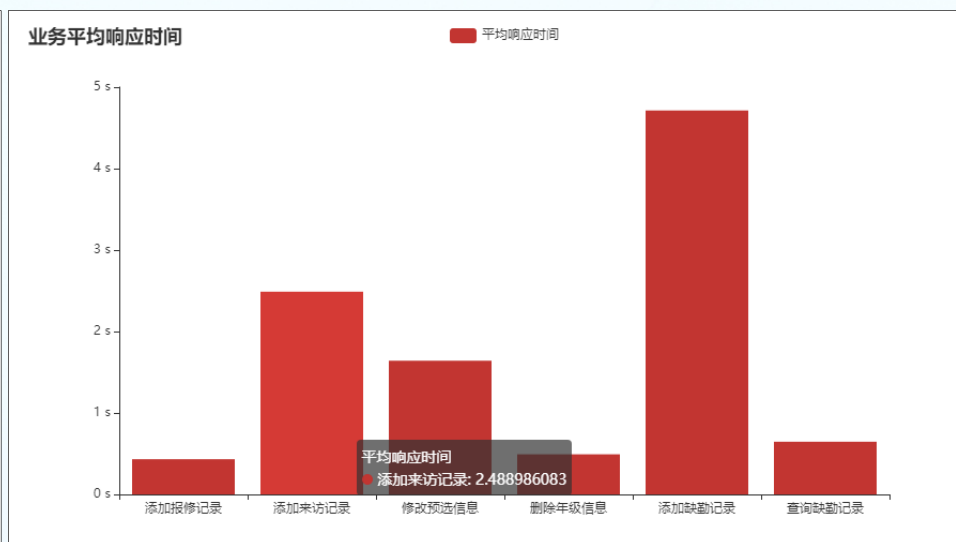




新使命·新研发
New Mission · New R&D



业务平均响应时间概况



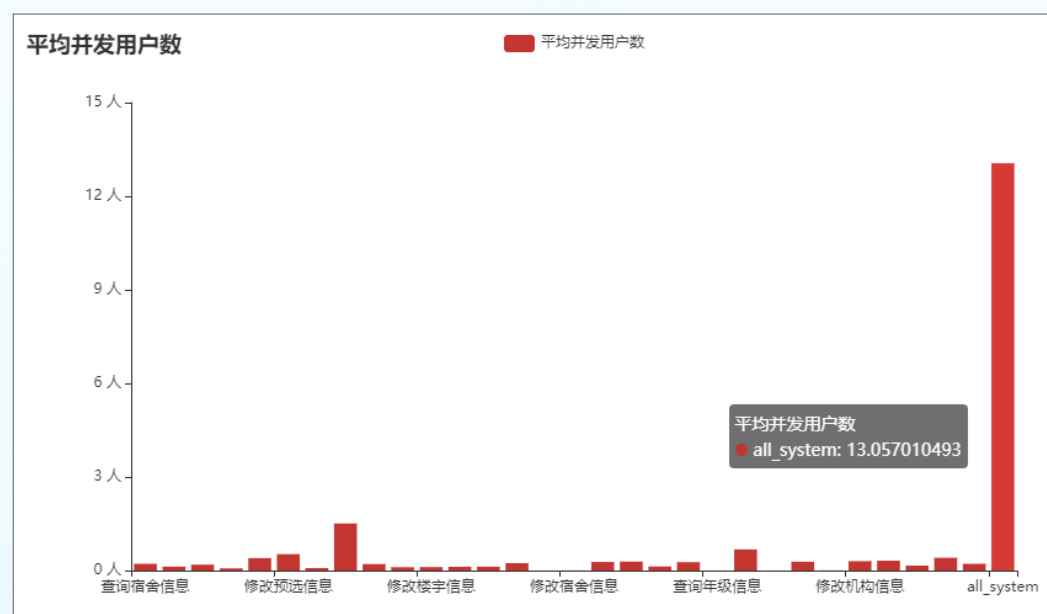
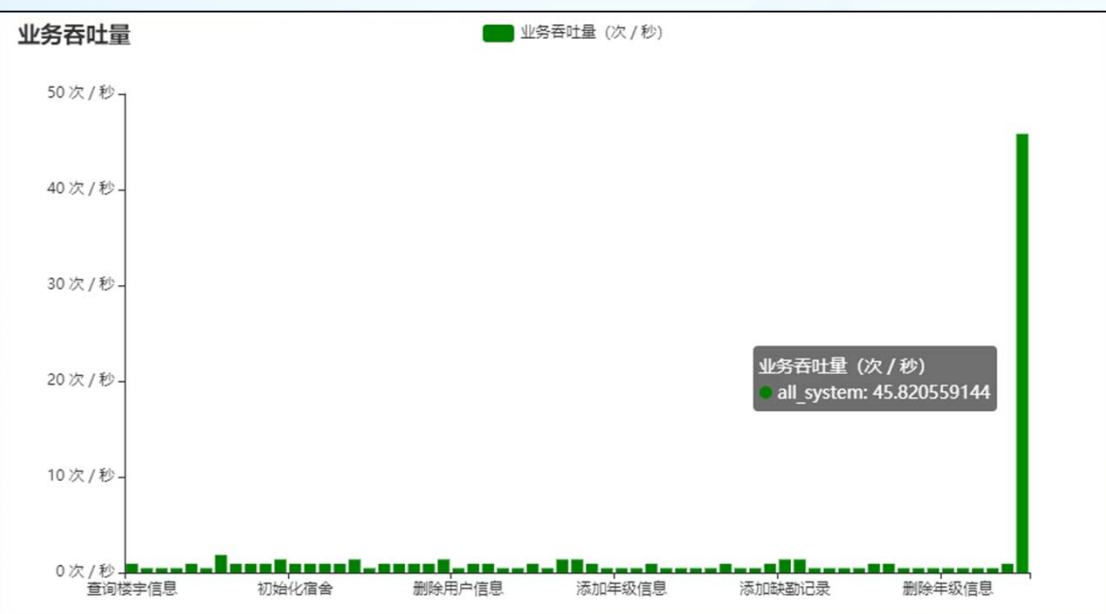
业务平均响应时间详情



新使命·新研发
New Mission · New R&D

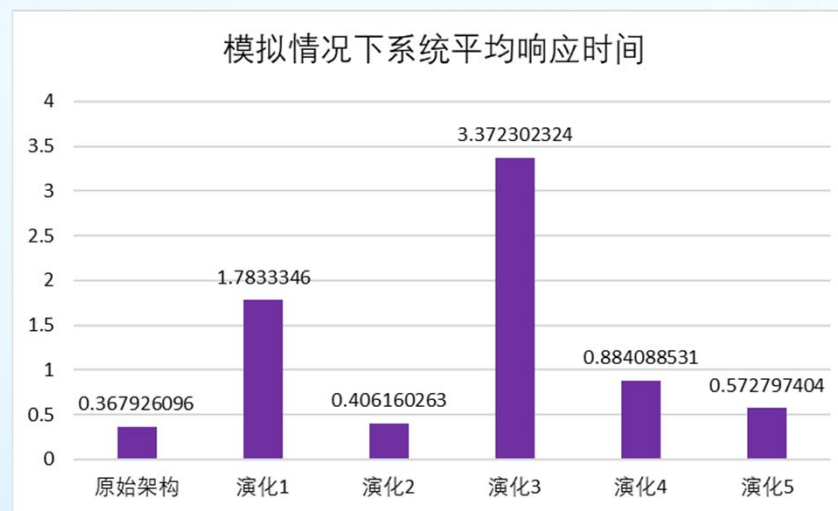
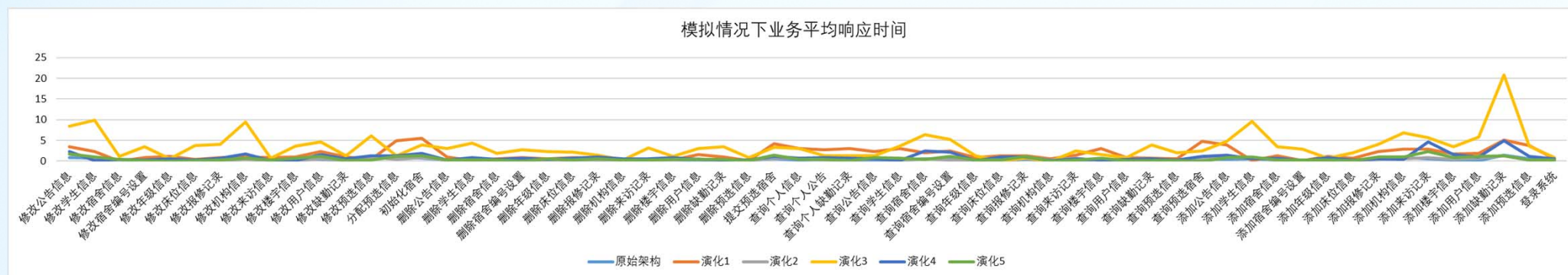
系统吞吐量展示

系统平均并发用户数展示



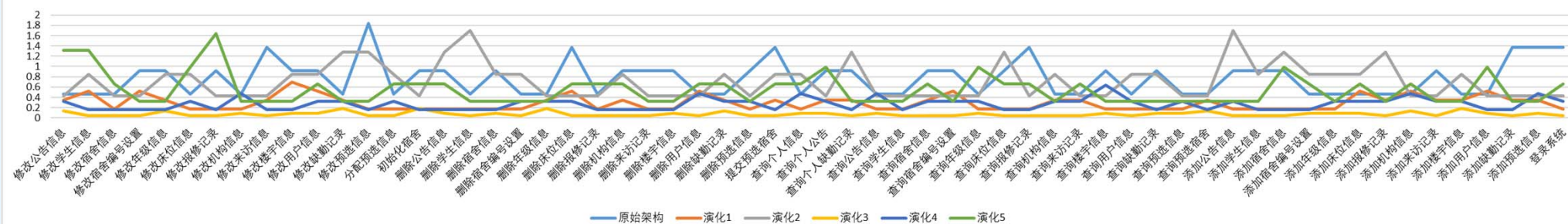


(3.4) 架构发生演化时系统性能对比

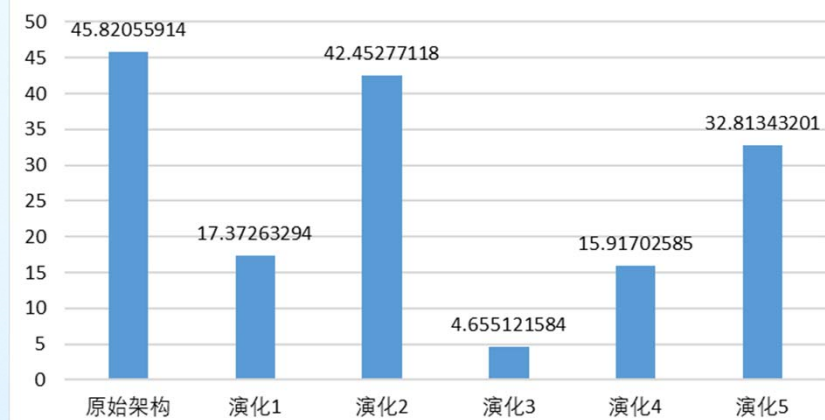


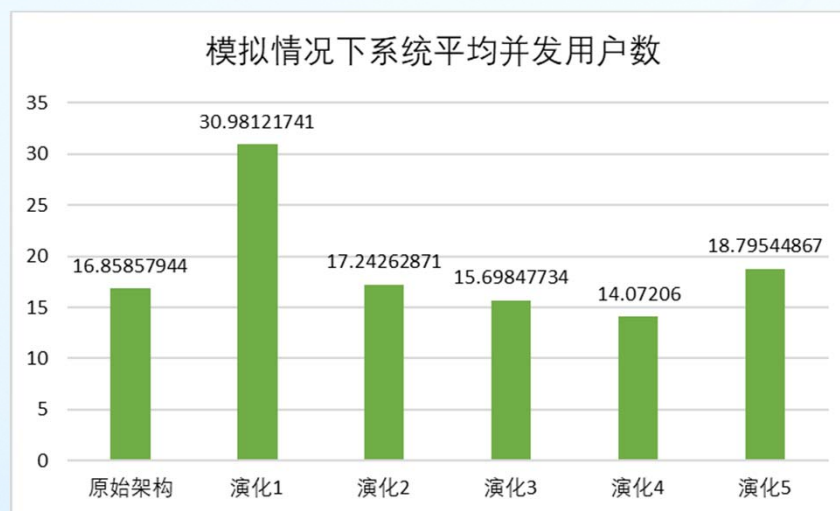
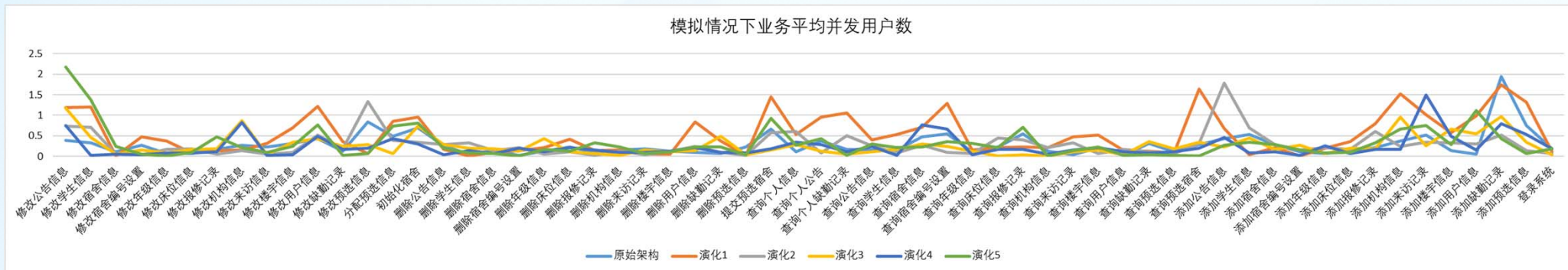


模拟情况下业务吞吐量



模拟情况下系统吞吐量







新使命·新研发
New Mission · New R&D

四. 算法原型

简介: [SA.Performance.Evaluator v1.0](#)是一个面向软件架构的性能评估系统, 主要针对UML建模的软件架构进行多方位的性能评估。SA.Performance.Evaluator v1.0通过计算设备响应时间、网络传输时间与程序响应时间实现对软件架构性能的静态分析; 通过模拟用户请求系统服务, 仿真系统实际运行情况, 计算业务平均响应时间、业务吞吐量与平均并发用户数, 动态对软件架构性能进行多方位评估。SA.Performance.Evaluator v1.0 旨在软件设计早期对软件架构性能进行评估, 为程序员提供一个多角度分析软件架构性能的方法, 帮助程序员尽早地发现软件架构性能存在的问题, 不仅可以预防风险, 还可以极大地降低软件后期的运维成本, 为高质量的软件设计提供了保障。



新使命·新研发
New Mission · New R&D

原型工具SA.Performance.Evaluator用户主界面





新使命·新研发
New Mission · New R&D

SA.Performance.Evaluator

模型文件选择

设备信息标注

业务信息标注

架构性标注

架构性能评估流程

1. 模型文件选择

2. 设备信息标注

3. 业务信息标注

4. 架构性能评估

未读入xml文件，文件

读入xml文件：D:\performance-test-dormitory.xml

提交

设备节点	CPU频率 (GHZ)	设备响应时间倍数(10^6)	设备最小操作响应倍数(10^6)	设备线程数
1 Client	4.0	1.0	1.0	5
2 DataBase	4.0	1.0	1.0	5
3 Execution Server	4.0	1.0	1.0	5
4 Interceptor Server	4.0	1.0	1.0	5
5 JAVA	4.0	1.0	1.0	5
6 Maven	4.0	1.0	1.0	5
7 WebUI	4.0	1.0	1.0	5

CPU频率：CPU每秒钟运算次数

设备响应时间倍数：设备响应所需要的CPU时钟周期数

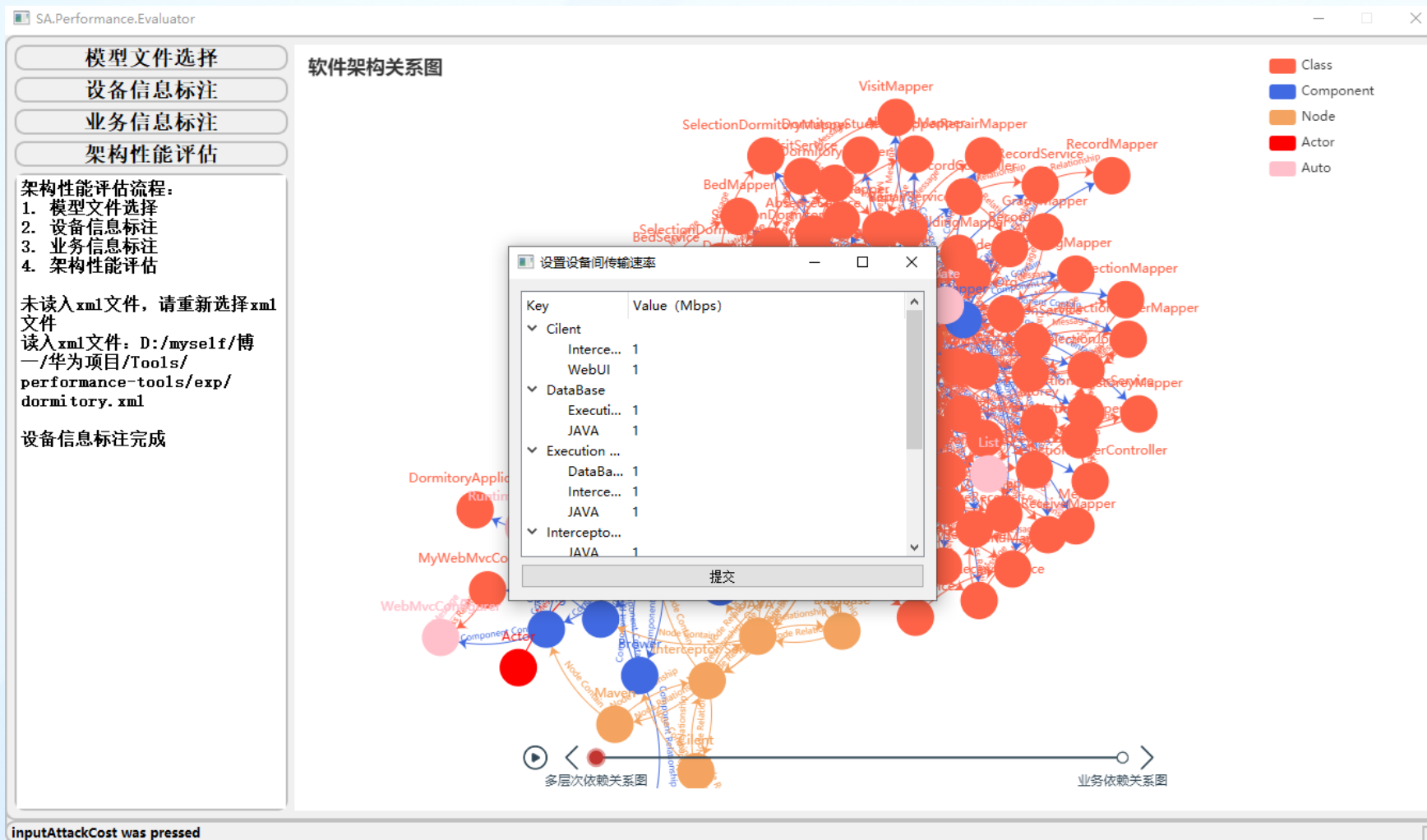
设备最小操作响应倍数：原子操作所需要的CPU时钟周期数

设备线程数：设备允许同时执行的任务操作数

inputAttackCost was pressed



新使命·新研发
New Mission · New R&D





新使命·新研发
New Mission · New R&D

SA.Performance.Evaluator

模型文件选择

设备信息标注

业务信息标注

架构性能评估

软件架构关系图

Class

Component

Node

Actor

Auto

业务信息标注

业务名称	业务执行频率	业务数据量 (Kb)	业务路径执行频率	业务操作执行时间
1 修改公告信息	1	1	1	0.002828427
2 修改学生信息	1	1	1	0.007148102
3 修改宿舍信息	1	1	1	0.002850439
4 修改宿舍编号设置	1	1	1	0.004506939
5 修改年级信息	1	1	1	0.003748123
6 修改床位信息	1	1	1	0.011288261
7 修改报修记录	1	1	1	0.002828427
8 修改机构信息	1	1	1	0.007148102
9 修改来访信息	1	1	1	0.003865623
10 修改楼宇信息	1	1	1	0.006144103
11 修改用户信息	1	1	1	0.004924429
12 修改考勤记录	1	1	1	0.004609772

修改公告信息

业务路径0

业务路径1

业务路径2

Axios.post(Map<String,String>)

TokenInterceptor.preHandle(HttpServletRequest, HttpServletResponse, Object)

NoticeReceiveController.query(NoticeReceive)

NoticeReceiveService.query(NoticeReceive)

NoticeReceiveMapper.query(NoticeReceive)

Result.ok(PagelInfo)

Axios.post(Map<String,String>)

TokenInterceptor.preHandle(HttpServletRequest, HttpServletResponse, Object)

BuildingController.query(Building, HttpServletRequest)

UserService.detail(Integer)

UserMapper.detail(Integer)

BuildingService.query(Building)

BuildingMapper.query(Building)

UserService.detail(Integer)

提交

多层次依赖关系图

业务依赖关系图

inputAssetValue was pressed



新使命·新研发
New Mission · New R&D

SA.Performance.Evaluator

模型文件选择

设备信息标注

业务信息标注

架构性能评估

架构性能评估流程:

1. 模型文件选择
2. 设备信息标注
3. 业务信息标注
4. 架构性能评估

未读入xml文件, 请重新选择xml文件
读入xml文件: D:/myself/博一/华为项目/Tools/performance-tools/exp/dormitory.xml

设备信息标注完成
设备信息标注完成
业务信息标注完成

软件架构关系图

模拟方式选择

☒ 用户到达时间模拟方式一

☐ 用户到达时间模拟方式二

设定用户总人数 (人)

100

设定用户到达频率 (人/s)

10

提交

Class

Component

Node

Actor

Auto

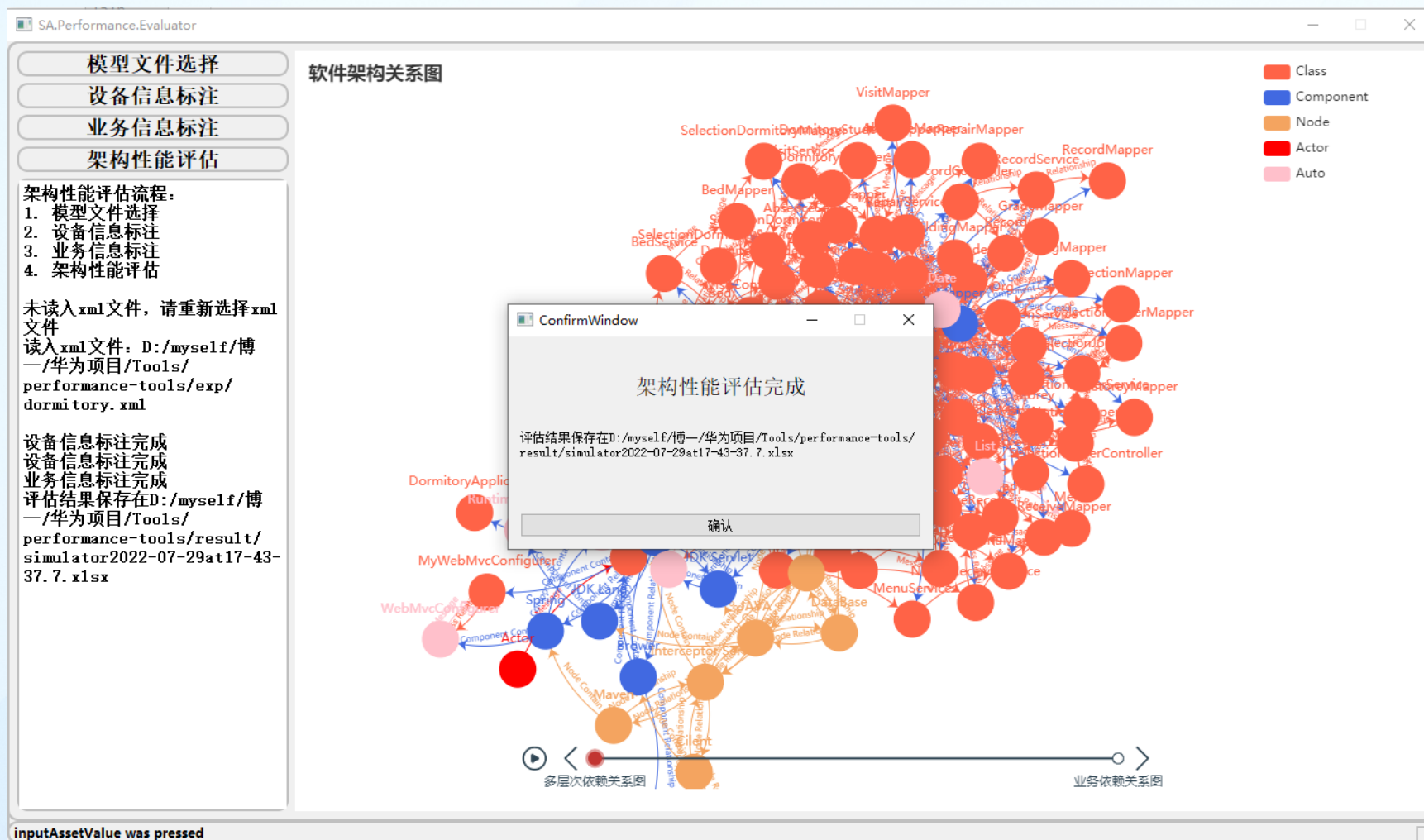
多层次依赖关系图

业务依赖关系图

inputAssetValue was pressed



新使命·新研发
New Mission · New R&D





新使命·新研发
New Mission · New R&D

五. 结束语

几点观察

- ① 内因(架构设计)决定性能、外因(环境)影响性能。【设计的核心作用+创造性】
- ② 架构设计需要考虑性能和其他属性(可扩展性、可靠性、安全性、韧性等)之间的**权衡**(Tradeoff)。【例如：最小路径可以用来权衡韧性和性能】
- ③ 性能水很深，No Silver Bullet！【计算方式+硬件+软件+使用上下文+环境】



新使命·新研发
New Mission · New R&D

• 需要考虑的问题

- ① 性能反模式
(Performance
AntiPattern)
是什么?



Automating Performance Antipattern Detection and Software Refactoring in UML Models

Davide Arcelli, Vittorio Cortellessa, Daniele Di Pompeo
Department of Computer Science and Engineering, and Mathematics
University of L'Aquila (Italy)
{davide.arcelli,vittorio.cortellessa,daniele.dipompeo}@univaq.it

- ② AI技术在性能
提升中能起什
么作用?



Machine Learning-Assisted Performance Testing

Mahshid Helali Moghadam
RISE Research Institutes of Sweden, Mälardalen University
Västerås, Sweden
mahshid.helali.moghadam@ri.se

Performance Prediction of Configurable softwares using Machine learning approach

Tanuja Shailesh
Computer Science and Engineering
Manipal Institute of Engineering, MAHE
Manipal, India
tanuja.s@manipal.edu

Ashalatha Nayak
Computer Science and Engineering
Manipal Institute of Engineering, MAHE
Manipal, India
asha.nayak@manipal.edu

Devi Prasad
Software Consultant
Manipal India
dprasadm@gmail.com

- ③ 混源软件的性
能如何估算?



Performance of GitHub Open-Source Software Project: An Empirical Analysis

Abin Joy, Senthilkumar Thangavelu, Amalendu Jyotishi
Amrita School of Business
Amrita Vishwa Vidyapeetham
Bangalore, India
abinjoy@gmail.com, senmalkisasb@gmail.com, amalendu.jyotishi@gmail.com

参考文献

1. Rajendra P. Mikkilineni, Donald F. Utter Jr.: *Designing software for maintenance and performance*. COMPSAC 1988, Chicago, IL, USA, 5-7 October, 1988, Pages 135-140.
2. Connie U. Smith. *Performance Engineering of Software Systems*, Addison-Wesley, 1990.
3. Connie U. Smith. *Origins of Software Performance Engineering: Highlights and Outstanding Problems*. Performance Engineering, LNCS 2047, pp. 96–118, 2001.
4. Connie U. Smith and Lloyd.G. Williams. *Performance Solutions: A Practical Guide to Creating Responsive, Scalable Software*. Addison-Wesley (2001).
5. David C. Roberts and David A. Grosman. *Modifying the Software Development Life Cycle to Include Software Performance Assurance*. WESCANEX 97.
6. Carlos Eduardo de Barros Paes. *RUP Extension For the Software Performance*. COMPSAC 2008.
7. Mitchell O. Locks, "*Relationship between minimal path sets and cutsets*", IEEE Trans. Reliability, vol R-27, No 3. 1978, p 106.
8. V. K. Bansal, K. B. Misra and M. P. Jain. *Minimal Pathset and Minimal Cutsets Using Search Technique*. Microelectron Reliab., Vol. 22, No. 6, pp. 1067-1075, 1982.
9. G. B. Jasmon, O. S. Kai. *A New Technique in Minimal Path and Cutset Evaluation*. IEEE Trans. Reliability, vol R-34, No 2, 1985, p 136-143.
10. Zi Li, Xiaoyang Li, Rui Kang. *System Resilience Modeling and Assessment based on Minimal Paths*. Prognostics and System Health Management Conference, Chenggdu, 2016.
11. Dave Garlan and Mary Shaw. *Software Architecture: Perspectives on an Emerging Discipline*, Pearson Press, 1996.
12.



新使命·新研发
New Mission · New R&D

THANK YOU